Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3766073

PDF Download
3766073.pdf
29 January 2026
Total Citations: 0
Total Downloads: 315

Accepted: 19 August 2025
Revised: 07 August 2025
Received: 09 February 2025

Citation in BibTeX format

RESEARCH-ARTICLE

More Than Meets the Eye: On Evaluating SBOM Tools In Java

MENGHAN WU, Zhejiang University, Hangzhou, Zhejiang, China

YUKAI ZHAO, Zhejiang University, Hangzhou, Zhejiang, China

XING HU, Zhejiang University, Hangzhou, Zhejiang, China

XIAN ZHAN, Huawei Technologies Co., Ltd., Shenzhen, Guangdong, China

SHANPING LI, Zhejiang University, Hangzhou, Zhejiang, China

XIN XIA, Zhejiang University, Hangzhou, Zhejiang, China

Open Access Support provided by:

Zhejiang University

Huawei Technologies Co., Ltd.

More Than Meets the Eye: On Evaluating SBOM Tools In Java

MENGHAN WU, Zhejiang University, China
YUKAI ZHAO, Zhejiang University, China
XING HU*, Zhejiang University, China
XIAN ZHAN, Huawei, China
SHANPING LI, Zhejiang University, China
XIN XIA, Zhejiang University, China

Open-source software is widely used in current software development. Unfortunately, this integration introduces a spectrum of challenges and potential threats. Such challenges emerge due to the diversity of import scenarios, which in turn may introduce malicious or vulnerable code in the client software, thereby causing significant security risks. To improve the transparency of software supply chains, Software Bill of Materials (SBOM) tools are proposed to identify the components within software systems. However, there limit investigation of functionality (i.e., tool operational process and data fields) and their practical performance of SBOM tools across various import scenarios. In this paper, we perform a comprehensive empirical study to investigate the impact of different import scenarios on SBOM tools. Specifically, we focus on three distinct component import scenarios: Build Tool Import, Dynamic Loading, and Source Code Import across a new benchmark consisting of 152 projects. We find that (1) The detection capabilities of SBOM tools exhibit considerable variance, especially in identifying dependency relationships; (2) The effectiveness of SBOM tools within the import scenarios of Dynamic Loading and Source Code Import falls short of expectations. Based on our findings, we summarize the lessons learned from different perspectives, including practitioners, tool vendors, and researchers. Our study provides valuable insights into the intricate landscape of software component usage, contributing to enhancing SBOM tools in modern software development.

CCS Concepts: • **Security and privacy** → **Software security engineering**; • **Software and its engineering** → **Risk management**; **Software libraries and repositories**.

1 INTRODUCTION

Nowadays, open-source software (OSS) has been widely used in modern software development. Linux Foundation reported that 90% organizations use OSS at a moderate, significant, or widespread level [70]. With the rapid growth of OSS usage, software becomes more complex and easier to be influenced with vulnerabilities [73, 84, 88]. These vulnerabilities are often propagated from reused OSS components to client software [71, 93, 102]. Specifically, within the Java ecosystem, the challenges associated with OSS reuse become notably prominent [59]. For example, Apache Log4j vulnerability (CVE-2021-44228 [41]) has impacted over 44% of Java projects. As a result, software

*Corresponding Author

Authors' Contact Information: Menghan Wu, Zhejiang University, Hangzhou, China, menghanwu@zju.edu.cn; Yukai Zhao, Zhejiang University, Hangzhou, China, yukaizhao2000@zju.edu.cn; Xing Hu, Zhejiang University, Hangzhou, China, xinghu@zju.edu.cn; Xian Zhan, Huawei, Hangzhou, China, chichoxian@gmail.com; Shanping Li, Zhejiang University, Hangzhou, China, shan@zju.edu.cn; Xin Xia, Zhejiang University, Hangzhou, China, xin.xia@acm.org.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2025 Copyright held by the owner/author(s).
ACM 1557-7392/2025/9-ART
https://doi.org/10.1145/3766073

Just Accepted

development grapples with the challenges inherent in the reuse of intricate components [69, 82, 95, 110]. Thus, it is important to analyze and manage the components imported into the software.

Software Bill of Materials (SBOMs) raise attention by providing detailed lists of components that make up software applications and their corresponding information like dependencies and licenses. Through a global access point, SBOMs can enhance the reusability, transparency, and security in modern software development [12, 39, 105, 120]. However, according to [20, 96, 111], creating a precise and comprehensive SBOM is challenging. Our comprehensive literature review (Section 2) highlights several challenges to SBOM generation. These challenges primarily concern the accuracy, efficiency, and comprehensiveness of component, dependency relationship, and vulnerability analyses [80, 96, 111, 116]. These insights underscore the need for a robust evaluation of SBOM tools to ensure they meet the evolving demands of software development.

Considering the prevalence of widespread component reusability [9], this need for SBOM evaluation is more significant in the Java ecosystem. Based on our preliminary study, current Java projects have three methods to import components: ❶ ***Build Tool Import (BTI)***, ❷ ***Dynamic Loading (DL)***, and ❸ ***Source Code Import (SCI)***. *BTI* is the default method due to the automatic download and management of building tools or IDE tools. For example, developers can download required third-party components through the Maven central repository [75], to simplify the build process. *DL* [90] enables the Java Virtual Machine (JVM) to load classes and resources at runtime, typically through dynamic class loading and reflection, enhancing adaptability and resource utilization. *SCI* [92] represents a typical way where the client projects directly use the source code of a particular third-party component, which enables more fine-grained customization of third-party component reuse. In practice, a project frequently includes a variety of import methods for software components.

Recent research on SBOM evaluation has primarily focused on two aspects, accuracy [15, 91, 115, 120] and functionality (i.e., tool workflows and supported data fields) [39, 56], across ecosystems, including Python, Ruby, PHP, and Java. Among these, accuracy-oriented studies are often constrained by the construction of ground truth. A representative example is the work by Balliu et al. [15], which evaluates six SBOM tools for Java projects using ground truth derived from Maven CLI outputs [47]. While this study provides valuable insights into the limitations of current tools and marks a step toward systematic evaluation, its reliance on Maven CLI consider components explicitly declared in build configurations, which omit components introduced via Source Code Import (SCI) or Dynamic Loading (DL) [18]. To overcome this limitation, our work extends ground truth construction by incorporating all three major import methods. Specifically, we combine build tool outputs with file-level parsing and targeted keyword-based detection (e.g., reflection patterns) to identify a broader set of components. Manual validation further ensures the completeness and accuracy. Moreover, prior works on SBOM functionality [39, 56] have largely focused on structural aspects such as supported metadata fields, with limited attention to tool performance in practical development scenarios. In contrast, our study complements existing literature by providing a more comprehensive ground truth and evaluating tool effectiveness on real-world Java projects.

In this paper, we conduct a systematic comparison of six state-of-the-art SBOM generation tools using two datasets that are both diverse and representative: Dataset 1 (*D1*) consists of real-world Java projects, while Dataset (*D2*) is carefully constructed to ensure balanced coverage across the three major component import methods (*BTI*, *SCI*, and *DL*). Our study aims to achieve two objectives: ① analyze and compare the functionality (i.e., tool operational process and data fields) from the source code and generated SBOMs, ② evaluate the practical effectiveness and efficiency of generating SBOMs on component and dependency relationship detection. These SBOM tools include `CycloneDX Generator (Cdxgen)` [38], `SBOM Tool (SbomTool)` [78], `SPDX SBOM Generator (SPDXGen)` [83], `OWASP Dependency Track (OWASP)` [50], `Syft` [10], and `Trivy` [94]. For the functionality of these tools, we focus on the operational processes (i.e., *artifact scanning*, *component detection*, *dependency resolution*, *license detection*, *vulnerability detection*, and *report generation*) as well as data fields. For practical performance evaluation, we introduce a new benchmark to assess the tools' detection capability under

Just Accepted

different importing scenarios. We evaluate these tools with three criteria: *Effectiveness*, *Time Efficiency*, and *Capability of Detection in Different Import Scenarios*.

The results show that the detection capabilities of SBOM tools vary substantially in components and dependencies. Moreover, the SBOM tools vary in different import scenarios. They perform better in build tool import scenarios than dynamic loading and source code importing. The findings shed light on crucial facets of SBOM tools, offering guidance for ongoing enhancements by encouraging developers to consider components in a wider range of import methods.

In summary, our main contributions are as follows:

- We comprehensively evaluate six SBOM tools from two aspects: the functionality (i.e., tool operational process and data fields), as well as practical effectiveness and efficiency in generating SBOMs.
- We conduct a comprehensive literature review about SBOM and summarize the current challenges of SBOM.
- We construct the first comprehensive benchmark with 152 Java projects, covering three component import approaches, including dynamic loading, build tool import, and source code import. This benchmark is also applicable for evaluating Software Composition Analysis (SCA) tools.
- Based on our analysis, we identify the shortcomings of existing tools and highlight the potential challenges in current SBOM tools. We suggest tool selection under different usage scenarios for developers and provide useful insights for future research on SBOM.

The rest of this paper is organized as follows. Section 2 details the SBOM background and related works. Section 3, we present detailed analyses of each SBOM tool's workflow and the comparison of data fields of each SBOM tool. Section 4 describes the preliminary study that underpins our benchmark design. We present the evaluation framework in Section 5. The evaluation results of our evaluations are discussed in Section 6, followed by implications and recommendations for future work in Section 7.

2 BACKGROUND AND RELATED WORKS

In this section, we provide our literature review about SBOMs and related works on evaluation.

2.1 Software Bill of Materials (SBOM)

A Bill of Materials (BOM) is a detailed list of the raw materials, components, and parts required to manufacture a final product [64]. With the increasing complexity of software supply chains and the growing emphasis on security, a similar concept called the Software Bill of Materials (SBOMs) has emerged in software management [96, 97]. It is used to identify and document the components and dependencies within the software, ensuring greater transparency and security in the software supply chain [27, 67, 96, 98]. While for different software systems, there are different BOMs (e.g., AIBOMs [30], DataBOMs [16], OBOMs [37], and HBOMs [36]). In this paper, we discuss SBOMs for software production. Software can be represented as a hierarchy of components, which may come from third-party sources or be developed internally. SBOMs provide a standard way to capture and present information about these components, According to the National Telecommunications and Information Administration (NTIA) [106], the minimum data fields parts of an SBOM component include: Supplier Name, Component Name, Version of the Component, Other Unique Identifiers, Dependency Relationship, Author of SBOM Data, and Timestamp. Three major SBOM format specifications currently exist: SPDX [51], CycloneDX [35], and SWID [12] in which SPDX and CycloneDX are mostly used [87].

Several studies have engaged directly with the stakeholders involved in SBOM implementation. For instance, studies by Zahan et al. [116], Xia et al. [111], Stalnaker et al. [96], and Kloeg et al. [67] utilized surveys and interviews to examine the current applications, benefits, and challenges associated with SBOMs. These studies highlight stakeholders' perspectives and explore the practical implications of SBOM integration across various

Just Accepted

industries. The exploration of the SBOM has garnered significant attention in recent years, contributing to a deeper understanding and enhancement of SBOM practices. Moreover, discussions from open-source communities such as GitHub and Stack Overflow have been analyzed to highlight the community's perception and the real-world challenges in software development. Studies by Bi et al. [20] and Otoda et al. [86] specifically focus on how SBOM practices are viewed and implemented within these communities, revealing gaps and opportunities for improvement.

Evaluations of SBOM tools have also been conducted to compare their capabilities in various aspects such as generation accuracy [15, 91, 115], sharing machinic [112], vulnerability detection [22, 65, 87], and support for different programming ecosystems [39, 56, 114]. These studies reveal significant performance variations and the need for improvements in tool accuracy and reliability. Additionally, efforts to improve SBOM tool robustness are illustrated in studies by Arora et al. [13] and Ding et al. [44], which proposed innovative methods to overcome the limitations of current tools and better support the security and transparency of software supply chains. Furthermore, several studies focused on SBOM adoption and integration into various domains, like those by Nocera et al. [80] and Zhou et al. [121], delved into the challenges and potential barriers to broader adoption, particularly exploring how SBOMs are applied in service-oriented contexts and different programming ecosystems.

In summary, the aforementioned studies have provided the advances and ongoing challenges in SBOM practices, laying a foundation for both future research and practical advancements in the field. We summarize these challenges across the SBOM lifecycle stages (i.e., Planning, Construction, Delivery, and Development) as detailed in Table 1. Our analysis reveals that a significant portion of the challenges revolves around managing SBOM components, **underscoring the critical importance of accurate and efficient component and dependency relationships in SBOM practices.** In this paper, we use 'dependency' to denote a dependency relationship. This highlights the need for robust tools capable of handling complex dependencies with high precision and reliability.

Given this context, our research aims to conduct a thorough evaluation of SBOM generation tools, focusing specifically on the tools' functionality (i.e., tool operational process and data fields) as well as their practical effectiveness and efficiency in generating SBOMs. This evaluation will help identify current practices and tool features in SBOM generation, thereby enhancing the overall security and transparency of software supply chains.

2.2 Tool Evaluation

In recent years, numerous empirical studies have addressed various aspects of evaluating Software Composition Analysis (SCA) tools [23, 40, 43, 60, 119, 120] and SBOM tools [15, 39, 56, 91, 115]. Zhao et al. [120] assessed the capabilities of six SCA tools in detecting dependencies and vulnerabilities across 13,708 real-world Maven projects. Similarly, Balliu et al. [15] examined the accuracy of SBOMs generated by six tools for complex open-source Java projects. However, these works are limited in their ground truth construction to the Maven dependency tree. Furthermore, other studies have focused on SBOM tools across various programming languages. Yu et al. [115] employed a differential analysis approach to evaluate generated SBOM differences without the ground truth for 7,876 open-source projects in languages including Python, Ruby, PHP, Java, Swift, C#, Rust, Golang, and JavaScript. Rabbi et al. [91] specifically targeted the npm ecosystem, analyzing SBOM tools on 50 JavaScript projects using the npm list command to establish ground truth. These studies don't construct the complete ground truth for components in projects. These studies often construct ground truth using package manager tools like Maven and npm but **overlook the complete components importing with various methods except by package managers in actual projects**.

Several works compare SBOM tools by analysing the information they generate [39, 56]. Halbritter et al. [56] provide the ground truth by presenting the ground truth by manually validating four open-source projects written in different languages and present the SBOM accuracy and completeness. Our study focuses solely on the Java

Just Accepted

Table 1. Challenges Across SBOM Lifecycle Stages

Stage	Challenge	Description
Planning	Systematic Design of SBOM Features [15, 20, 96, 111]	Developing a comprehensive and standardized approach for defining SBOM features across various use cases.
	Risk Identification [20]	Identifying potential risks and vulnerabilities that might arise from SBOM implementation and usage.
	SBOM Documentation Preparation [20, 80, 116]	Assessing and preparing detailed SBOM documentation to ensure compliance and clarity.
	Compliance Integration [20, 116]	Integrating SBOM standards with existing compliance frameworks to streamline adoption.
	Complexity of Specifications [96]	Addressing the challenges posed by the complexity and variability of SBOM specifications.
	Standards Incompatibility [96, 111]	Resolving incompatibility issues between different SBOM standards for seamless integration.
Construction	SBOM Generation [20, 96, 111]	Efficiently generating SBOMs during the software build process, ensuring accuracy and completeness.
	Quality Assessment and Verification [20, 111]	Ensuring the accuracy, quality, and reliability of generated SBOMs.
	Insufficient Tooling [96, 116]	Limited availability of effective tools for SBOM creation, leading to manual effort and inefficiencies.
	Legacy Package Support [20, 96]	Developing methods to generate SBOMs for legacy software packages and repositories.
	Absence in OSS [80, 111]	Highlighting the need for incentives to encourage SBOM adoption in widely used open-source software.
	Vulnerability Disclosure for Limited Exploitability [15, 111]	Careful construction of vulnerability disclosures within SBOMs to handle limited exploitability issues.
Delivery	Secure Delivery [20, 111]	Ensuring secure and reliable delivery of SBOMs across the software supply chain.
	Keeping SBOMs Up to Date [15, 80, 96, 116]	Regularly updating SBOMs to reflect ongoing changes in software components and versions.
	Verifying Accuracy and Completeness [96, 116]	Confirming the accuracy and completeness of SBOMs to maintain trust and reliability.
	Time Consumption [116]	Managing the time and resources needed to create, verify, and maintain SBOMs.
Development	SBOM Data Integration [20]	Integrating SBOM data into existing systems, processes, and tools to enhance utility.
	Ecosystem Differences [15, 96]	Addressing the variations across different software ecosystems to ensure SBOM compatibility.
	Completeness vs. Privacy Trade-off [96, 116]	Balancing the need for comprehensive SBOM data with concerns over data privacy and security.
	Tool Maturity and Interoperability [111, 116]	Developing more reliable and interoperable SBOM tools to address maturity gaps.

Just Accepted

language and utilizes a much larger and more diverse dataset consisting of 132 real-world Java projects, thereby deepening the empirical findings and implications for Java SBOM generation. Zhang et al. [119] provided an

empirical comparison of published dependency remediation strategies by various SCA tools and introduced CORAL, which offers remediation recommendations that maintain compatibility. Other studies [40, 43, 60] have explored the reachability of detected vulnerabilities by SCA tools, yet they have not delved into the accuracy of component detection, which forms the core functionality of these tools. Given these gaps, our study aims to conduct a comprehensive evaluation of SBOM tools, specifically for the Java ecosystem, assessing not only the tools' functionality but also the practical effectiveness and efficiency in generating SBOMs.

2.3 SBOM Usage Scenarios

Existing studies have demonstrated that SBOMs are applicable in a wide variety of practices [34, 96, 113]. Building on these studies, we extract the common application scenarios of SBOM and summarise the corresponding requirements on SBOM tooling.

1. CI/CD Pipeline Integration. In modern DevOps workflows, SBOM generation must be **fully automated, lightweight, and fast** to avoid slowing down the build. Tools offering incremental or delta analysis (e.g., JFrog Xray's[1] deep recursive scan with delta support) enable real-time dependency detection without compromising development [42, 68]. Consequently, **Requirement 1** – *SBOM tools should generate incremental results fast enough to avoid becoming the bottleneck of a CI/CD pipeline and provide native plugins or actions for common build systems such as Maven, Gradle, and GitHub Actions.* For example, the CycloneDX Maven plugin is embedded in many pipelines to generate an SBOM on every commit, ensuring continuous visibility into component changes [61].

2. Vulnerability Management. Security teams rely on SBOMs to enumerate **all** components and match them against vulnerability databases such as NVD [79] or CVE [33]. Therefore, **Requirement 2** – *SBOM tools must provide comprehensive component list and precise versioning metadata to maximise vulnerability detection recall and reduce false negatives.* Effective tools also support the integration with vulnerability scanners [24, 49]. One industry report [24] shows that SBOMs with precise versioning reduce mean time to detect vulnerabilities by 37% in enterprise development.

3. License Compliance and Auditing. Legal and compliance teams use SBOMs to verify third-party license compatibility (e.g., GPL v3 [52] vs. MIT [74]). Thus, **Requirement 3** – *SBOM tools must accurately extract and normalise licence identifiers, outputting SBOM with SPDX or CycloneDX formats suitable for auditing.* For instance, Fossa[2] focuses on early-stage license detection and reports incompatible licenses during the development lifecycle [48, 62].

4. Regulatory and Procurement Requirements. Government and industry regulations (e.g., US EO 14028 and EU Cyber Resilience Act) increasingly demand SBOM submission for critical software. Accordingly, **Requirement 4** – *SBOM tools must emit artifacts that strictly conform to demanded formats such as NTIA Minimum Elements and support all required fields.* Failure to meet the format disqualifies software from procurement in regulated environments [45, 104].

5. Third-Party Risk Assessment. Enterprises performing supply chain risk analysis use SBOMs to map **explicit and hierarchical dependency graphs** and identify vulnerabilities from transitive dependencies. Therefore, **Requirement 5** – *SBOM tools must preserve the full dependency tree (including transitive dependency relationships) to enable impact or exploitability analysis across the graph.* A case study [29] reports that organizations leveraging graph-rich SBOMs reduced third-party risk exposure by 29% through comprehensive dependency monitoring.

[1] https://jfrog.com/xray/
[2] https://fossa.com/

Just Accepted

3 SBOM TOOL COMPARISON

In this section, we provide our tool selection and evaluate SBOM tools' functionality including tool operational process and data fields.

3.1 Tool Selection

To ensure our selection of SBOM tools is systematic, reproducible, and directly relevant to the research community, we adopted a two-stage selection process. This approach is designed to ground our evaluation in the existing literature, thus avoiding an ad-hoc selection of tools. **Stage 1: Identifying the Candidate Tools from Academic Literature.** First, to identify academically relevant candidate tools, we followed the study by [120] to conduct a literature review. Our goal was to find tools that have been previously evaluated or utilized in existing research. We applied the following inclusion criteria for studies: ② Selected studies are published in respected venues, including journals ranked in the first quartile by `SJR`[3] (Q1) and conferences rated A or B under `CORE`[4] or `CCF`[5] recommendation. Through this process, nine relevant papers [15, 39, 40, 56, 60, 89, 91, 115, 120] are identified. **Stage 2: Filtering for Final Tools Selection.** From the candidate tools identified in the literature, we selected the final set of tools for our evaluation based on three strict criteria tailored to our research objectives: ① Capable of generating SBOMs, as our study focuses specifically on SBOMs rather than other output forms; ② Mentioned in at least two of the selected papers, reflecting sufficient research attention and validation; ③ Usable as command-line tools to support automatic experiments and reproduction. This two-stage process resulted in the selection of six tools: `CycloneDX Generator (Cdxgen)` [38], `SBOM Tool (SbomTool)` [78], `SPDX SBOM Generator (SPDXGen)` [83], `OWASP Dependency Track (Dependency Track)` [50], `Syft` [10], and `Trivy` [94]. We utilize the latest stable releases (accessed on December 1, 2023) of these tools. The details of our used SBOM tools are shown in Table 2.

3.2 Tool Comparison

We introduce the process of SBOM generation and compare the state-of-the-art SBOM tools.

3.2.1 Overview of SBOM Generation Process. The basic SBOM generation process contains six steps, which are explained below.

Step 1: Artifact Scanning. SBOM tools start by scanning the local project repository of the target software. This involves parsing the target artifacts to identify package structures, components, and configurations. There are three strategies for identifying third-party component candidates: ① identify the components in configuration files [28, 109]; ② consider all the independent Java archives like "*.jar" as component candidates [63, 107]; ③ conduct code clone detection algorithms to get independent components as candidates [17, 117].

Step 2: Component Detection. This step focuses on recognizing and extracting software components. In Java, software components are typically identified in GAV (GroupId, ArtifactId, Version) format. SBOM tools identify the software components: ① by validating them online using the component features like SHA value [19, 66]; ② by verifying whether the component candidates are present in the local `.m2` repository to retrieve the official GAV from Maven Central [25, 26]; or ③ by comparing the similarity of the codebase within the beforehand ground truth in tools [58]. This includes both components explicitly mentioned in the codebase and potential transitive dependencies.

[3]https://www.scimagojr.com/journalrank.php
[4]https://core.ac.uk/
[5]https://www.ccf.org.cn/

Just Accepted

Table 2. Free and Open-source SBOM Tools

Tool	Forks	Stars	Format	Description
SbomTool [78]	101	1.3k	SPDX 2.2	Quickly produces SPDX 2.2-compatible SBOMs using package scanning during build time. It also serves as a library for detecting dependencies in custom applications.
Syft [10]	449	4.9k	CycloneDX, SPDX	A CLI tool and library generating SBOMs from container images and filesystems, excellent for vulnerability detection with Grype.
Trivy [94]	1.9k	19.2k	CycloneDX, SPDX	Adapts its file analysis based on target types—code repositories or binary artifacts like container images to ensure effective security analysis.
SPDXGen [83]	106	334	SPDX v2.2	A command-line tool that creates SPDX SBOMs using current package managers, aligning with SPDX v2.2 specifications, including NTIA's recommended elements.
Cdxgen [38]	101	347	CycloneDX	Creates CycloneDX SBOM for projects from source and container images. Supports many languages and package managers.
Dependency Track [50]	492	2100	CycloneDX	Minimizes software supply chain risks by monitoring component usage across applications, integrating seamlessly into CI/CD environments.

Just Accepted

Step 3: Dependency Parser. The step usually engages **Step 2** in resolving relationships among components. This entails determining the specific versions and configurations of each component and understanding their relationships with other components. The goal is to accurately and comprehensively parse the project's dependency tree. Dependencies include both direct and indirect dependencies. Direct dependencies are those explicitly declared in the project's configuration files, while indirect dependencies are those required by the direct dependencies themselves.

Step 4: License Detection. License detection is a key process that involves identifying the licenses under which components are released. This step ensures compliance with legal requirements and helps organizations manage their license obligations effectively. License detection is particularly important for avoiding legal risks associated with proprietary and open-source software usage.

Step 5: Vulnerability Detection. SBOM tools detect the components for known security vulnerabilities. This process utilizes various databases and vulnerability feeds to check vulnerabilities. It is crucial for maintaining software security and integrity, as it enables developers to take timely action against potential exploits.

Step 6: Report Generation. The results are then compiled into a detailed report in specific formats like SPDX 2.2 [51] that meet different purposes for different stakeholders.

3.2.2 Comparison of SBOM Generation Process. We introduce and compare the state-of-the-art SBOM tools with details shown in Table 3 and Table 4 by manually inspecting their source code and generating SBOMs.

Scanning Method Comparison. SBOM tools typically scan three types of artifacts: ① `POM` files, ② archive files (i.e., `JAR` files), and ③ Maven dependency tree after build. Among these, `Trivy`, `Syft`, `Cdxgen`, and `OWASP` are the most prevalent tools for scanning archive files. Only `Trivy` and `Cdxgen` scan `POM` files in projects. `Cdxgen` scans all kinds of artifacts, while `OWASP` only scans `JAR` files.

Table 3. Comparison of SBOM Tool Capabilities

Workflow	Artifact/Approach	SbomTool	Trivy	Syft	SPDXGen	Cdxgen	Dependency Track
Artifact Scanning	pom.xml		★			★	
	Archive		.jar, .war, .ear, .par	.jar, .war, .ear, .lpkg, .par, .sar, .nar, .kar		.jar, .war, .ear, .par	.jar, .war
	Maven Dependency Tree	★			★	★	
Component Detection	maven cli	★			★	★	
	parse Pom.xml	★	★	★	★	★	★
	parse Archive files		★	★			★ (ignore nested .jar files)
	remote API					★	
Dependency Parser	Direct	★	★	★	★	★	
	Indirect		★	★	★	★	
	Parse maven tree	★			★	★	
	pom parser		★	★	★	★	★
License Detection	ClearlyDefined API	★					
	parser from database		★	★			
	parser from pom and archive	★	★	★	★	★	★
Vulnerability Detection	provided		★				★
	search through dB		★				★
	search through external API						★
Report Generation	SPDX	★	★	★	★		★
	CycloneDX		★	★		★	★

Just Accepted

Component Detection Method Comparison. SBOM tools utilize three approaches to detect components: ① through Maven CLI, ② through a POM parser, and ③ through an archive parser. Some tools like `Syft` are capable of parsing both `POM` and archive files. In contrast, `Trivy` and `OWASP` only parse pom files within archive files. Particularly, `Trivy` can parse nested jar files, whereas `OWASP` cannot handle nested jars.

Dependency Parser Method Comparison. Not every tool focuses on accurately resolving dependencies in SBOMs. Specifically, `SbomTool` and `OWASP` do not provide methods for parsing transitive dependencies. Other tools, however, do offer methods to resolve these indirect dependencies, which are typically identified through two approaches: ❶ by scanning Maven dependency tree, and ❷ by parsing `POM` files within the project. `SPDXGen` and `Cdxgen` implement both retrieval methods. Other tools implement only one of these methods. For instance, `SPDXGen` identifies direct dependencies within the `POM` files and transitive dependencies through Maven dependency tree. The process of parsing build files involves scanning specified paths for Maven (`POM.xml`), Gradle (`build.gradle`), Bazel, or SBT (`build.sbt`) files and parsing the dependency information contained within them. For example, if direct file parsing does not yield complete dependency information, `Cdxgen` executes corresponding build tool commands to extract the dependency tree. These commands enable the program to obtain both direct and transitive dependencies.

License Detection Method Comparison. All tools retrieve license information based on data from `POM` files and archive files. For accurate license information, these tools process data by querying third-party APIs or local databases. Specifically, `SbomTool` retrieves license information by querying third-party APIs, whereas `Trivy` uses a local database for license detection.

Vulnerability Detection Method Comparison. Not all tools offer vulnerability detection capabilities. Typically, tools that do provide this function perform vulnerability checks on component information after retrieving it. Only `Trivy` and `OWASP` conduct vulnerability detection. Vulnerability detection is conducted through databases and POM files. Specifically, `Trivy` performs vulnerability detection solely through database queries, while `OWASP` uses both databases and third-party APIs for vulnerability searches.

Report Comparison Three tools support two standard SBOM formats: `Trivy`, `Syft`, and `OWASP`; while others support only one format of three official SBOM formats. Besides, in this paper, we do not consider the self-defined SBOM formats.

Table 4. Data Fields of SBOMs from Evaluated Tools

Category		SbomTool	Trivy	Syft	SPDXGen	Cdxgen	Dependency Track
Essential	Author Name	★	★	★	★	★	★
	Timestamp	★	★	★	★	★	★
	Supplier Name	★	★	▼	★	★	★
	Component Name	★	★	★	★	★	★
	Version	★	★	★	★	★	★
	Component Hash	☆	☆	SHA1, SHA256	SHA1	MD5, SHA1, SHA256, SHA384, SHA512, SHA3-256, SHA3-384, SHA3-512	MD5, SHA1, SHA256
	Unique Identifier	SPDXID	bom-ref, SPDXID	bom-ref, SPDXID	SPDXID	bom-ref	purl
	Relationship	☆	★	★	★	★	☆
	Licenses	▼	★	▼	▼	★	★
Provided	Vulnerabilities	☆	▼	☆	☆	☆	★
	Download Location	▼	purl	▼	Maven Url	purl	Ossindex Url
	Files	★	★	★	☆	☆	★
	Relationship Type	DESCRIBED_BY, DESCRIBES, DEPENDS_ON	DEPENDS_ON	DESCRIBES, CONTAINS, OTHER	DEPENDS_ON	☆	☆
	Component Type	☆	application (pom.xml), library	☆	★	library, framework	☆

Note: ★ indicates that the data field has been recognized. ▼ indicates that the data field is provided but not recognized. ☆ indicates no data available.

3.2.3 Comparison of Data Fields in SBOMs. Table 4 shows the data fields comparison of SBOM tools. According to [106], the essential data fields of SBOMs are *Author Name*, *Timestamp*, *Supplier Name*, *Component Name*, *Component Hash*, *Version*, and *Unique Identifier*. `SbomTool` and `Trivy` lack the component hash field but include all other data fields. The remaining tools encompass all essential data fields. This comparison builds upon and extends prior work such as [56], which focuses on identifying the existence of certain fields. Our analysis explores their detailed semantics and representation. Table 4 incorporates additional fields critical to Java projects and software supply chain risk assessment (e.g., vulnerability metadata support and component verification information). Furthermore, the table distinguishes format-specific characteristics to assess compatibility for Java ecosystems. This provides the necessary information for the subsequent evaluation on the performance of SBOM tools.

Regarding the provided data fields, our comparison reveals that *License Detection* and *Vulnerability Detection* vary widely among the tools. For instance, while the `SPDXGen` does not directly recognize certain licenses, it effectively parses project files and archives to infer licensing information. On the other hand, `OWASP` excels in *Vulnerability Detection* by integrating data from both internal databases and external APIs, thus offering robust security insights. Furthermore, the capability to parse relationship types such as *DEPENDS_ON*, *DESCRIBED_BY*, and *CONTAINS* is inconsistently implemented across the tools. This feature is essential for understanding software composition.

Just Accepted

Overall, while all the analyzed tools offer a core set of SBOM capabilities, differences in their approaches to *Component Hashes*, *Vulnerability Detection*, and *Relationship* parsing emphasize the distinct focuses and strengths of each tool. Users seek to select an SBOM tool that best aligns with their specific needs in terms of security, compliance, and management of open-source software components. However, due to discrepancies in the data fields across different SBOM tool reports, comparing these reports and parsing the resulting data presents significant challenges. Consequently, evaluating current SBOM tools is crucial for both academic research and industry applications.

4 PRELIMINARY STUDY

In this section, we conduct a preliminary study aimed at exploring two preliminary questions (PQs) that inform the design of our subsequent benchmark and evaluation framework.

PQ1 (Inclusion Criteria): What types of components should be included in an SBOM? This question explores and defines which components qualify for inclusion in an SBOM, addressing the current ambiguity in SBOM component definitions. Exploring this will help establish clear guidelines for what constitutes the content of an SBOM.

PQ2 (Import Methods of Software Components): How are components imported into Maven projects? This question seeks to explore the various methods through which components are imported into Maven projects. By analyzing these import methods, we aim to understand the diverse development scenarios and patterns utilized in Maven. It allows developers and project managers to evaluate Maven's suitability for their specific projects more efficiently and leverage its capabilities to fulfill their requirements efficiently.

4.1 Approach

To systematically investigate the preliminary questions, we first study existing literature to understand how software components are defined within SBOMs in *PQ1*. To address *PQ2*, we then extend our research to analyze how these components manifest within real-world Java projects managed with Maven. We download 952 Maven projects in the top 2k GitHub projects. There are three potential components in Maven projects according to [14, 85, 101], components in POM files, JAR files, and custom components. For POM files, we identify the components by Maven CLI. Then, we search for JAR files in current projects and manually analyze their usage to understand their inclusion within the projects. Furthermore, we explore the incorporation of source code, examining both the integration of custom modules and the inclusion of external modules like git submodules.

4.2 Results

4.2.1 PQ1: Inclusion Criteria for Components in SBOMs. To establish the inclusion criteria for components in SBOMs, we first revisit the definition of a software component. According to [108], a software component is defined as "a unit of composition with contractually specified interfaces and explicit context dependencies, capable of independent deployment and susceptible to third-party composition." This definition underscores the fundamental characteristics of components. Furthermore, components are typically imported as JAR files and source code [85, 101]. This can ensure that both third-party libraries (TPLs) and internal developed components are effectively utilized in a project. According to the SBOM definition in Section 2, an SBOM must include both TPLs and internal developed components. TPLs are often sourced from well-established repositories such as Maven repository [46], GitHub [54], and Bitbucket [21]. Internal developed components refer to the components that developers often import directly from the source code to customize functionalities or to create new components that meet their specific needs. This practice is also particularly important for addressing cross-language import scenarios, as Maven cannot manage packages from different programming languages effectively. For example, project `questdb` [5] has submodules `jemalloc` [3] and `simdjson` [7]. These two components are C libraries that

Just Accepted

cannot be imported by Maven. By using source code directly, developers can ensure compatibility and functionality across diverse development scenarios.

> **Answer to PQ1:** SBOM components should include both TPLs and internal developed components. These components are typically integrated using JAR files or source code, allowing for customization and addressing Maven's limitations. Specifically, many package managers (e.g., Maven) cannot effectively manage cross-language packages, necessitating alternative methods for importing components to ensure successful project implementation.

4.2.2 PQ2: Import Methods of Software Components. To investigate the component import methods, we manually analyze the usage of JAR files and source code components across 952 collected Maven projects.

Table 5. Various Import Methods for Open-source Software Components.

Import Method	Existing Approaches	#Projects	Description
Build Tool Import	By Maven, Gradle, etc.	952	This method uses GAV identifier coordinates to accurately locate and retrieve third-party components from the official central repository. [77]
	By local project JAR import	21	Leveraging Maven's build management capabilities, developers can efficiently manage and include locally stored JAR files through POM files [85].
Dynamic Loading	By reflection, classLoader, etc.	43	Dynamic loading in Java refers to using classes and components at runtime instead of loading them during the static compilation phase [90].
Source Code Import	By using the `.submodules.git` file	38	The `.submodules.git` file facilitates the integration of external code repositories, enhancing modularity and simplifying dependency management by recording submodule configurations, including remote repository URLs and specific commit states [53].
	By importing source code directly	✗	This approach involves directly incorporating source code files into the project's codebase, eliminating external dependencies and improving control over the integrated code. [92, 99]

✗ indicates that we do not consider this import method in our study due to its specificity and rarity.

As shown in Table 5, we summarize three methods concerning importing software components in Java projects (including *Build Tool Import (BTI)*, *Dynamic Loading (DL)*, and *Source Code Import (SCI)*) as well as their distribution. *BTI* is employed in every project of our dataset. *DL* is employed in 43 projects, demonstrating its relevance for runtime flexibility. In contrast, *SCI* is less common, with 38 projects opting for this approach, highlighting its specific use cases in managing external code repositories and enhancing control over dependencies. This distribution reflects the varying needs and methods developers utilize when importing components into their Java applications.

Build Tool Import (BTI). This method utilizes build tools like Maven [46] or Gradle [55] or IDE tools to automatically import software components. Dependencies are specified in the POM file to automatically download and manage them from a central repository. *BTI* ensures consistency and simplifies management by enforcing adherence to predefined project structures. **Our study exemplifies the *BTI* using Maven.** Developers may also need to manually download/develop, configure, and import software components that are often provided as *jar* files, especially when these components originate from private repositories or are unavailable for automatic import due to network constraints. This process requires careful attention to dependency management and version compatibility.

Just Accepted

Dynamic Loading (DL). *DL* allows developers to import software components during program execution. Java supports this through various APIs and mechanisms, such as JAVA reflection.

Source Code Import (SCI). *SCI* requires directly importing the source code of software components into a project. This involves integrating external codebases as submodules or packages into the client project. *SCI* provides a high level of control and transparency, allowing developers to synchronize their project's code repository with specific versions and customize the functionality of external components. However, this approach requires careful consideration of the structures of components and their potential architectural impact on the project as a whole. There are two common methods for *SCI*, as indicated in Table 5, by utilizing the ".submodules.git" file and by directly importing the source code.

> **Answer to PQ2:** There are three main methods for importing software components in Java projects: *Build Tool Import*, *Dynamic Loading*, and *Source Code Import*. The distribution of these import methods is unbiased.

5 EVALUATION FRAMEWORK

In this section, we present the overview of our evaluation framework, evaluation criteria, and benchmark construction.

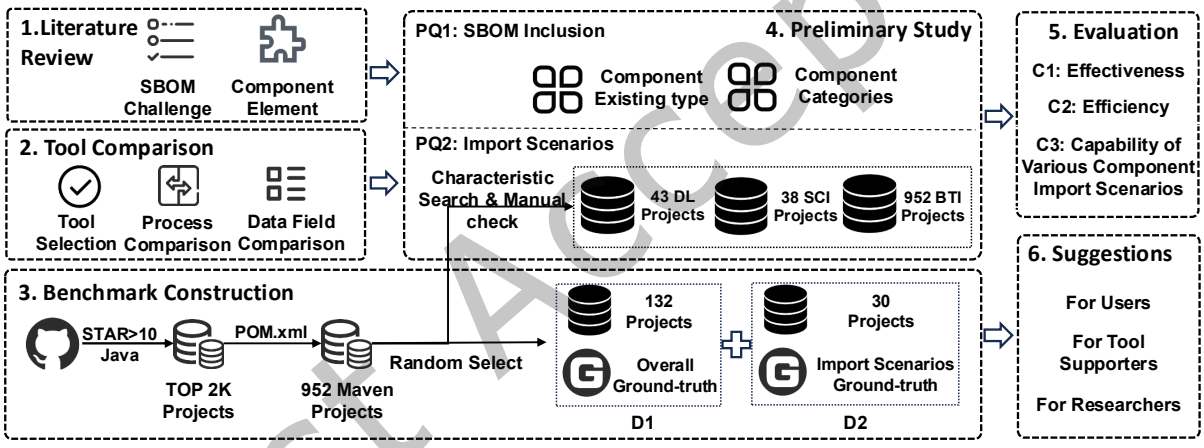

Fig. 1. Overview of Our Evaluation Framework.

5.1 Overview

Figure 1 shows an overview of our framework, which aims to comprehensively evaluate SBOM tools for Maven-based Java projects. Initially, we conduct a literature review in Section 2 to identify the prevailing challenges associated with SBOM components. We find that most challenges are directly related to the components within SBOMs. Furthermore, upon examining existing evaluation works for SBOM tools, we notice a significant gap: there is a lack of thorough comparisons that address both the operational processes of SBOM tools and their practical effectiveness and efficiency in SBOM generation. To address these shortcomings, we propose a dual-focused evaluation strategy. First, we analyze the operational processes of six open-source SBOM tools (i.e., `CycloneDX Generator`, `SBOM Tool`, `SPDXSBOM Generator`, `OWASP Dependency Track`, `Syft`, and `Trivy`) by examining their manuals and source codes. Each tool's approach to source code scanning, component detection, dependency resolution, license detection, vulnerability detection, and report generation is already compared in detail in Section 3.

Just Accepted

Additionally, we propose a new benchmark that consists of projects employing single import methods, allowing for a precise evaluation of SBOM generation's effectiveness and efficiency. This new benchmark was motivated by our preliminary study detailed in Section 4, which highlighted the variability in component import methods and the inadequacies of existing evaluations to assess tool performance across these variations effectively.

5.2 Evaluation Criteria

We aim to conduct a comprehensive evaluation of state-of-the-art SBOM tools from a practical perspective. To accomplish this, we have designed an empirical study using the following criteria:

C1 (Effectiveness of Detection): We compare the effectiveness of component detection and dependency detection of existing tools on a unified dataset (without bias) from the real world. The evaluation is based on three key metrics: recall, precision, and F1 score.

C2 (Time Efficiency): Efficiency is critical when evaluating SBOM tools, especially concerning their scalability and suitability for industrial software. We measure the time each tool takes to generate an SBOM using the same dataset to assess which tools are capable of handling large-scale operations effectively.

C3 (Capability of Detection in different Import Scenarios): Based on our preliminary study, components within projects are often imported using diverse methods. This confuses the detection result in different import scenarios. Therefore, we aim to assess the capability of each SBOM tool in accurately identifying components imported through different methods.

5.3 Benchmark Construction

For the evaluation benchmark, we collect two datasets for different purposes: ❶ Detecting Components and dependencies in open-source projects (e.g., from GitHub) to evaluate the effectiveness/efficiency of each tool in the real world (*C1 & C2*). ❷ Detecting Components and dependencies in constructed projects with single import methods to evaluate the capability of each tool in different import scenarios (*C3*). We utilize a distinct dataset for evaluating *C3* due to limitations in the first dataset: ① **Lack of scenario labeling.** The SBOM tools used do not specify the import scenarios for detected components, making it unclear which scenarios the components belong to. This is crucial for our analysis. ② **Imbalance in real project scenarios.** Our preliminary study shows that the distribution of import scenarios in real projects is uneven, which could bias our evaluation results. Therefore, a separate dataset is necessary to ensure a balanced and accurate assessment of *C3*.

5.3.1 Dataset for Overall Effectiveness/Efficiency Evaluation (D1). D1 contains 132 projects sourced from real-world software. It needs to satisfy two requirements: ① including popular open-source projects, along with accurate ground-truth. ② providing the components with the strict Group, Artifact, and Version (GAV) format to ensure uniformity and precision. The ground-truth consists of the components (C) detailed with their import scenarios (I), and dependencies (D). The scenario label (I) is used for ground-truth construction that accurately reflects the scenarios in which each component is utilized. For example, within a given project, a component c ($c \in C$) has import scenarios I_c (where each $i_c \in I_c$, and $I_c \subset \{BTI, SCI, DL\}$). Additionally, this component c has dependencies D_c ($d(c, j) \in D_c$, and both $c, j \in C$). We use the GAV format to identify component c with versions and suppliers, which is essential for linking various vulnerability information in software security.

Project Collection: We first query the GitHub API, listing the top 2,000 Java projects with over 10 stars for high-quality projects. Second, we focus on Maven build projects as it is one of the widely adopted build automation tools for Java projects [120]. We collected 952 projects. To balance statistical rigor with practical feasibility in data collection and analysis (e.g., reducing the effort required for manual ground-truth (GT) construction and project analysis), we further random select a statistically valid sample size of 274 projects using Cochran's sample size formula [32], which is widely used in population proportion estimation [8, 57, 81]. This formula is applied with

Just Accepted

a 95% confidence level (i.e., we are 95% confident that the true population parameter lies within the calculated interval) and a 5% error margin (i.e., the maximum allowable difference between the sample estimate and the true population proportion). Finally, we successfully build 132 projects by following the configuration instructions provided in their `README` files, forming the final dataset for analysis.

Ground-truth Collection We construct the ground-truth by identifying the different components' import methods. We first label the components with their import methods (I). ❶ For components labeled with *DL*, we employ the filter process on our initial dataset, targeting projects with dynamic loading keywords. Specifically, we have two groups of keywords to present the dynamic loading process. The first group is keyword lists to link JAR files, including {`URLClassLoader` & `new URL`}, {`JarURLConnection` & `new URL`}, and {`import java.util.jar.JarFile`}. The second group aims to search keywords that load the class, including {`loadClass` & `newInstance` & `invoke`} and {`forName` & `newInstance` & `invoke`}. This process uses a series of keyword groups that describe the process of dynamic loading. The search terms are composed of two groups of keywords arranged in sequence (i.e., search term $= g1 \bigcap g2$, where $g1 \in group1$ and $g2 \in group2$). This process resulted in the identification of 10 projects. Then the first two authors manually check whether the third-party components are imported by dynamic loading. Finally, we obtain 10 projects with 48 components imported by dynamic loading. ❷ For components labeled with *SCI*, we check the existence of `.gitmodules` file in each project and finally found 2 projects with 3 components imported by source code. ❸ For components imported by Maven, we run the Maven CLI to gain the Maven dependency tree and extract the components with GAV format. We get 132 projects with 10980 components. Ultimately, we add detailed information about the corresponding dependencies for each identified component, thereby establishing a comprehensive ground-truth dataset of components and their dependencies. ❹ For dependency relationship construction, we construct a complete and structured dependency tree by integrating component metadata with their resolved dependencies across three dimensions: (i) Declared Components. We extract explicitly declared components from each module's original `pom.xml` file, capturing direct relationships specified by the developer. (ii) Transitive Resolution. We resolve full transitive dependency chains using Maven's build engine, which applies dependency mediation rules (e.g., nearest-wins, scope filtering) to produce the effective classpath for each module. (iii) Module Hierarchy Awareness. In multi-module Maven projects, each submodule is typically defined within a parent `pom.xml`. We treat each module as an individual component and reconstruct the hierarchical module structure to maintain clear boundaries between components and avoid conflating dependencies across different scopes. By combining these aspects, we generate an accurate and hierarchically organized ground-truth dataset that mirrors how modern build tools resolve dependencies. This structured representation enables precise tracing of component origins, enforces scope correctness, and supports fine-grained evaluation of SBOM generation tools.

5.3.2 Dataset for Detection Capability in Various Import Methods (D2). In order to investigate the capability of detection in different scenarios, we employ the dataset containing 30 projects with 3 sets, each containing components that utilize only one specific import method. We carefully developed ten projects each for *Dynamic Loading* (DL) and *Source Code Import* (SCI) using the popular components listed in Maven Central from different categories such as *JSON Libraries*, *HTTP Clients*, and *XML Processing*. Furthermore, to evaluate the *Build Tool Import* (BTI) scenarios, ten projects were randomly selected from D1 without any components labeled with *DL* or *SCI*. Since the BTI set is drawn from D1, there are 10 overlapping projects between D1 (132 projects) and D2 (30 projects), meaning D1 and D2 together comprise 152 unique projects in total. Finally, we get three sets (10×3) of projects with components in different import scenarios. We identify the components and corresponding dependencies of each project as our ground-truth in *D2*.

In total, *D2* includes 30 projects, covering a diverse range of component import strategies. Among them, the ground-truth includes a total of 2,030 components and 3,476 dependencies. To ensure a fair comparison across different import methods, we calculate the number of true positives (TP), false positives (FP), and false negatives

Just Accepted

(FN) for each individual project, and then compute the average values across all 30 projects. Consequently, the resulting values presented in Table 8 and Table 9 may appear as decimals rather than integers. This averaging approach provides a more stable comparison by mitigating potential bias from project size or dependency depth, especially considering the variability in component count across different import methods.

5.4 Experiment Configuration

We conduct our evaluation on two datasets consisting of 152 projects. Each project is analyzed using six SBOM tools with configurations described in their `README` file. We also present these configurations in Table 6. To avoid contamination from files written in other programming languages (e.g., `C` or `Python`) that are not relevant for Java third-party library detection, we implement a post-processing step to extract the components and dependency relationships from the generated SBOMs. Specifically, we save only Java components by identifying entries that include valid GAV (Group, Artifact, and Version) coordinates in SBOMs, which are standard identifiers in Java package management systems such as Maven and Gradle. We also extract dependency relationships from the item relationships presented in the collected SBOMs.

For all effectiveness evaluations (i.e., component detection and dependency detection), we calculate the metrics including true positives (TP), false positives (FP), and false negatives (FN) for each projects and present the average values in Table 8 and Table 9. Similarly, the derived metrics (i.e., Precision, Recall, and F1-score) are also computed individually for each project and then averaged to present a comprehensive and fair comparison across tools.

All experiments are conducted on five servers running Ubuntu 20.04, each equipped with an 18-core Intel Xeon CPU @ 2.30GHz and 256GB of RAM.

Table 6. onfiguration of Free and Open-source SBOM Tools in Experiments

Tool	Version	Output Format	Configuration Command
Syft	v0.91.0	SPDX-JSON	`syft <project-path> -o spdx-json > <output-path>/syft.spdx.json`
Trivy	v0.45.1	CycloneDX-JSON	`trivy fs -format cyclonedx -output <output-path>/trivy.result.json <project-path>`
SbomTool	v1.6.2	SPDX-JSON	`sbom-tool generate -b <output-path> -bc <project-path> -pn example -pv version1 -ps sbom-tool`
Cdxgen	v9.9.1	CycloneDX-JSON	`cdxgen -t java -o <output-path>/bom.json -required-only`
SPDXGen	v0.0.15	SPDX-JSON	`spdx-sbom-generator -format JSON -o <output-path>`
Dependency Track	v9.0.8	JSON	`dependency-check.sh -format JSON -project <project-name> -noupdate -prettyPrint -scan <project-path> -out <output-path>`

6 RESULTS

We present the evaluation results in this section.

Just Accepted

6.1 C1: Effectiveness

We evaluate the effectiveness and efficiency of selected SBOM tools using a dataset of 132 Maven projects. To evaluate the effectiveness, we first analyze all SBOM reports and identify the components and dependencies in SBOMs. Next, we compare these identified components and dependencies to the ground truth without considering the import methods. Then, we calculate the overall averages of the evaluation metrics, including false positives (FP), false negatives (FN), true positives (TP), Precision, Recall, and F1-score to evaluate existing tools. Table 7 presents the evaluation results of effectiveness for component and dependency detection across various SBOM tools.

Table 7. Performance Metrics of Different Tools for Component and Dependency Analysis.

Tool	Component			Dependency			Time (s)
	Precision	Recall	F1-Score	Precision	Recall	F1-Score	
SbomTool	93.72%	92.17%	92.39%	0.00%	0.00%	0.00%	48.32
Syft	46.33%	44.29%	41.11%	0.00%	0.00%	0.00%	5.09
Trivy	71.67%	42.04%	47.26%	46.00%	32.86%	32.81%	75.24
SPDXGen	71.43%	80.68%	74.99%	26.01%	43.46%	32.23%	29.88
Cdxgen	87.16%	85.16%	83.06%	47.13%	71.16%	54.57%	315.19
Dependency Track	65.67%	32.43%	37.01%	0.00%	0.00%	0.00%	49.88

6.1.1 Component Detection. As illustrated in Table 7, the performance of SBOM tools varies significantly in terms of F1-scores. `SbomTool` has the highest performance, achieving an F1-score of 92.39%, followed by `Cdxgen` and `SPDXGen`, at 83.06% and 74.99%, respectively. Instead, `Dependency Track` exhibits the poorest results with F1-scores of 37.01%, followed `Syft` at 41.11%. The better performance of `SbomTool`, `Cdxgen`, and `SPDXGen` can be attributed to their utilization of Maven CLI for component detection, allowing them to effectively leverage Maven dependency tree. In contrast, `Syft` and `Dependency Track` rely on parsing `JAR` files, which can result in incomplete component identification due to the limitations of archive file parsing.

False Negative Analysis. After the in-depth analysis of incomplete components, the FN is mainly caused by the following reasons. First, Maven cannot identify the third-party components by *DL* and *SCI* resulting in a limitation in identifying components as archive files and source code. Second, tools such as `Syft` and `Dependency Track` which scan artifacts (e.g., `JAR` files) within projects are also unable to identify components imported through source code. Table 2 unveiled scanning artifacts of SBOM tools. For instance, `Trivy` focuses on components published in official central (i.e., Maven Central [76]), often overlooking local `JAR` files used in projects. In contrast, `Syft` prioritized `JAR` files in projects, potentially missing components imported by maven `POM` files. These different detection approaches lead to the incompleteness of component detection.

False Positive Analysis. Two primary factors affect FP of SBOM tools: (1) conflicts in identifiers specifically GAV, and (2) inaccurate metadata in repositories. These conflicts stem from the different dependency parser strategies employed by various tools. Some tools directly parse `POM` files to extract version information for Maven projects. However, this approach has limitations, particularly when `POM` files do not explicitly specify version numbers for certain components. Tools relying heavily on `POM` file parsing may struggle to determine component versions in such cases accurately. For instance, in the project `mavenmvnd`[4], developers use a predefined property `jline.version`, which `Syft` fails to identify the correct version as `3.24.0`. Additionally, issues with repository metadata, such as outdated or abandoned dependencies in `POM` files, can significantly affect the accuracy of SBOM tools. Moreover, SBOM tools struggle to identify multiple versions of the same component accurately. In multi-module projects, different modules can use different versions of the same third-party component. This limitation underscores the need for SBOM tools to develop more robust detection approaches to handle these scenarios.

Just Accepted

6.1.2 Dependency Detection. As shown in Table 7, Cdxgen significantly outperforms other tools in dependency detection, achieving an F1-score of 54.57%. However, others fall short of 50%, highlighting a widespread issue with dependency detection across most evaluated tools. Among these, SbomTool, Syft, and Dependency Track perform the worst. Both SbomTool and Dependency Track focus exclusively on component detection, neglecting the dependencies that are crucial for comprehensive analysis. Additionally, Syft's reliance on JAR file parser leads to missing critical dependencies.

False Negative Analysis. The FN problem refers to missing dependencies of SBOM tools. The primary factors are incorrect component versions and the absence of components. For example, the missing component `org.apache.rocketmq:rocketmq-common:5.2.0` in the project rocketmq-spring[6] leads to the failure to recognize the dependency between `org.apache.rocketmq:rocketmq-common:5.2.0` and `com.alibaba:fastjson:1.2.83`, as noted by Trivy. This example illustrates how missing components directly affect dependencies identification. Our finding highlights the strong correlation between the capabilities of component detection and dependency detection in SBOM.

False Positive Analysis. FP manifests as inaccurate dependencies, significantly impacting the recall of dependency detection. In some cases, 'invented' dependencies can arise due to misinterpretation of code structures or the incorrect application of dependency parser approaches. In projects that rely on external third-party components bundled into the final JAR file after the build, some SBOM tools such as SbomTool and Trivy may overlook the dependency hierarchy within these fat JAR files. This limitation prevents these tools from effectively distinguishing between direct and transitive dependencies. For instance, in project Artillery[1], Trivy introduced a dependency ⟨`org.example:artillery:1.0-snapshot`, `com.jfoenix-jfoenix-8.0.4`⟩ that is not present in our ground truth. This results in an incorrect link between the project and the component `com.jfoenix-jfoenix-8.0.4`. Additionally, the wrong detection of components also causes a worse recall of dependency detection. For instance, when analyzing the fabric-sdk-java[2] project using Cdxgen, the component identifiers conflict with ground truth and also cause the failure to identify the dependency between component `org.hyperledger.fabric-sdk-java:fabric-sdk-java:2.2.27-SNAPSHOT` and `com.google.errorprone:error_prone_annotations:2.24.1`. This example points out that dependency parser approaches also play an important role in dependency identification.

> **Conclusion to C1:** SBOM tools have different capabilities to detect components and dependencies. While these tools excel in component detection, there remains significant room for improvement in dependency accuracy. SbomTool leads with an F1-score of 92.39% in component detection, while Cdxgen outperforms other tools in dependency detection, achieving an F1-score of 54.57%. Some SBOM tools (i.e., SbomTool, Syft, and Dependency Track) may prioritize component detection over in-depth dependency analysis. Component detection affects dependency detection among SBOM tools.

6.2 C2: Time Efficiency

To address *C2*, we conduct a comparative analysis of the time efficiency on *D1*. All tools were executed under identical conditions using their command-line interfaces to ensure fairness. The primary metric (time cost) evaluated was the duration required for the component detection process within each tool, as shown in Section 3.2. We report the average time cost across multiple projects for each tool to assess its practical runtime performance.

Table 7 presents the average time cost in six selected SBOM tools, facilitating a comprehensive comparison of their performance. Among these, Syft demonstrates the fastest average detection time at 5.09 seconds. This efficiency can be attributed to Syft's approach of directly scanning binary artifacts without invoking build tools like Maven[6]. By analyzing filesystems and archives (e.g., .jar and .war) without executing build commands, Syft

[6]https://github.com/anchore/syft/blob/main/syft/pkg/cataloger/java/cataloger.go

Just Accepted

avoids the overhead associated with dependency resolution processes, leading to quicker SBOM generation. In contrast, Cdxgen shows significantly longer detection times, averaging 315.19 seconds. This delay can be attributed to Cdxgen's reliance on search components from Maven Central[7] to extract comprehensive dependency information, including transitive dependencies. Such operations inherently consume more time due to the need for project compilation and network access to resolve dependencies. Additionally, Cdxgen integrates with Dependency-Track [8], a platform designed for intelligent component analysis, which involves remote API calls that further contribute to the increased processing time [9]. Both Dependency Track and SbomTool utilize API searches for vulnerability and license detection, which contributes to their slower processing times. Instead, Trivy, while relying on a local database for vulnerability and license searches, also experiences longer detection times due to the inherent complexity of the search processes involved. These observations are consistent with our earlier analysis in Table 3, which was derived from manual inspection of the tools' source code and execution behaviors.

Differences in speed and accuracy fundamentally shape SBOM tooling strategies. Syft represents a speed-optimized approach; however, our evaluation shows that its component-level F1-score is only 0.41. Moreover, based on our dependency relationship model, which is aware of module hierarchies in multi-module Maven projects, Syft fails to recover any dependency edges. At the opposite extreme, Cdxgen takes about the average of 315s per project but achieves the highest F1-scores in both component (83.06%) and dependency (54.57%) analysis. This makes it more appropriate for security-sensitive environments such as finance or healthcare. Tools like SbomTool (Time: 48.32s, Component Detection F1-score: 92.39%) and SPDXGen (Time: 29.88s, Component Detection F1-score: 74.99%) represent intermediate trade-offs, offering viable options for medium-sized projects that require a balance between speed and accuracy. These tools illustrate how time efficiency, when evaluated alongside accuracy, can support informed tool selection across different usage scenarios.

> **Conclusion to C2:** Syft outperforms existing SBOM tools regarding detection time, averaging only 5.09s per project, but performs worse in accuracy. Cdxgen takes significantly more time, approximately 315.19 seconds per project, but performs better in accuracy. This contraction suggests that developers should balance speed and completeness when selecting an SBOM tool. The detection scan methods employed by these tools significantly affect their efficiency. Tools that avoid build processes and remote API calls tend to perform faster, whereas those that involve comprehensive dependency resolution and remote integrations incur additional time costs.

6.3 C3: Capability of Various Component Import Scenarios

We evaluate the capabilities of the six selected SBOM generation tools toward various component import scenarios: ❶ *Build Tool Import (BTI)*, ❷ *Dynamic loading (DL)*, and ❸ *Source Code Import (SCI)*. We aim to investigate how these import scenarios influence the detection performance of each tool and to identify which scenarios have the most significant impact. This detection is conducted on *D2*. The results on components and dependencies are presented in Table 8 and Table 9.

We first compare the performance of SBOM tools by using the F1-score. For *SCI*, Trivy, Cdxgen and SbomTool exhibit the best performance with an F1-score of 48.08%. Specifically, SPDXGen does not detect any correct components in this scenario. In the *Dynamic Loading (DL)* scenario, Dependency Track reached an F1-score of 78.06%, while SPDXGen again failed to detect any components. For *BTI*, SbomTool achieves a perfect F1-score of 99.33% followed by Cdxgen at 99.28%, while Dependency Track and Syft show significantly lower F1-score

[7]https://github.com/CycloneDX/cdxgen/blob/de9c715c755c18824c715f06b35d00b998b9ab15/lib/helpers/utils.js
[8]https://github.com/DependencyTrack/dependency-track/tree/master
[9]https://github.com/CycloneDX/cdxgen/blob/12f6969d0c7a3823066a52c839720e984621b3d1/lib/cli/index.js#L8319

Just Accepted

Table 8. Effectiveness on Component Detection of Each SBOM Tool. Each column represents the different import methods: DL (Dynamic Loading), SCI (Source Code Import), and BTI (Build Tool Import).

Tool	DL						SCI						BTI					
	FP	FN	TP	Precision	Recall	F1	FP	FN	TP	Precision	Recall	F1	FP	FN	TP	Precision	Recall	F1
SbomTool	0	4.8	1	100.00%	22.63%	35.51%	0	3	1	100.00%	35.30%	48.08%	0.4	0.3	138.9	99.26%	99.40%	99.33%
Syft	10.2	1.1	4.7	39.68%	75.97%	50.30%	1.1	3.1	0.9	42.50%	32.80%	33.58%	96.7	48.7	90.5	53.61%	59.40%	52.93%
Trivy	0	4.8	1	100.00%	22.63%	35.51%	0	3	1	100.00%	35.30%	48.08%	0.1	100	39.2	69.90%	48.15%	54.05%
SPDXGen	1	5.8	0	0.00%	0.00%	0.00%	1	4	0	0.00%	0.00%	0.00%	17.6	6.9	132.3	81.29%	95.08%	85.80%
Cdxgen	0	4.8	1	100.00%	22.63%	35.51%	0	3	1	100.00%	35.30%	48.08%	0.4	0.5	138.7	99.26%	99.31%	99.28%
Dependency Track	2.4	1.1	4.7	86.76%	77.02%	78.06%	0.1	3.1	0.9	85.00%	32.80%	43.61%	13.8	57	82.2	85.60%	49.15%	56.79%

Table 9. Effectiveness on Dependency Detection of Each SBOM Tool.

Tool	DL						SCI						BTI					
	FP	FN	TP	Precision	Recall	F1	FP	FN	TP	Precision	Recall	F1	FP	FN	TP	Precision	Recall	F1
SbomTool	0	7.1	0	0	0	0	0	3.1	0	0	0	0	0	197.1	0	0	0	0
Syft	7.5	7.1	0	0	0	0	1	3.1	0	0	0	0	188.7	197.1	0	0	0	0
Trivy	0	7.1	0	0	0	0	0	3.1	0	0	0	0	33.2	159.8	37.3	47.32%	45.27%	40.62%
SPDXGen	0	7.1	0	0	0	0	0	3.1	0	0	0	0	182.8	69.3	127.8	31.67%	55.55%	40.05%
Cdxgen	0	7.1	0	0	0	0	0	3.1	0	0	0	0	172.9	9.8	187.3	57.73%	96.23%	71.78%
Dependency Track	0	7.1	0	0	0	0	0	3.1	0	0	0	0	0	197.1	0	0	0	0

at 56.79% and 52.93%. The evaluation reveals significant variation in tool performance across various import scenarios, suggesting that no single tool is capable of effectively managing all types of imports.

Considering the precision in Table 8, for *SCI*, `SbomTool`, `Cdxgen` and `Trivy` exhibit high precision of 100%. However, `SPDXGen` shows considerably lower precision scoring 0. According to our ground truth, we include `JAR` files generated after the build, which naturally favored tools capable of detecting jar files or those able to execute Maven CLI commands, resulting in a precision rate of 100% for these tools. `SPDXGen` does not explicitly provide the correct project supplier (GroupID) leading to its precision of 0. For *DL*, `SbomTool`, `Cdxgen`, and `Trivy` achieve the highest precision of 100%, while `SPDXGen` shows the lowest precision. With build tools, `SbomTool` and `Cdxgen` exhibit the highest precision. Instead, `Syft` and `Dependency Track` generate numerous false positives, indicating a need for improvements in their detection algorithms.

Regarding recall rates in *SCI* scenarios, `Trivy` and `SbomTool` exhibit the highest recall at 35.30%, while `SPDXGen` fails to detect any components. For *DL*, `Syft` and `Dependency Track` can perform better than others at 75.97% and 77.02%. Through manual analysis, we find that all tools are unable to detect components imported through source code; they can only identify jars that are generated and included in the ground truth post-compilation. For *BTI*, `SbomTool` and `Cdxgen` achieve the highest recall at 99.40% and 99.32%, while `Dependency Track` and `Trivy` score the lowest at 48.15% and 49.15%. This lower recall for `Dependency Track` is due to its inability to recognize circular jar dependencies. `Syft`, which only recognizes jar files, also shows a reduced recall because the jars it detects might be "thin jars", which do not contain all dependencies.

Regarding dependency detection, only `Trivy`, `SPDXGen`, and `Cdxgen` successfully identify dependencies in *BTI* scenarios, whereas other tools show no performance. In *DL* and *SCI* scenarios, none of the tools demonstrate any effective performance in detecting dependencies. Our findings suggest that it's important to select SBOM tools that align closely with the specific needs and import practices of a project to ensure optimal performance and security compliance.

Just Accepted

Conclusion to C3: Tool performance varies significantly across different import scenarios. There is no single tool fully capable of effectively handling all types of import scenarios, highlighting the need for tools to better adapt to varied import methods.

7 DISCUSSION

In this section, we provide the implications and threats in this paper.

7.1 Implications

We provide the implications for different stakeholders in this subsection.

7.1.1 From the perspective of users. We discuss current free and open-source SBOM tools about depth and propose suggestions in tool selection, guiding stakeholders on selecting the right tools for specific needs.

Scenario-Aligned Tool Selection. ❶ **CI/CD Pipeline Integration.** In CI/CD Pipelines, the quicker tool that still provides reliable results. `SPDXGen` completes a scan in about 30s with solid component coverage, making it ideal for per-commit checks. When vulnerability information is also needed, `Trivy` offers a comparable time (76s) while enriching the SBOM with vulnerability data. ❷ **Vulnerability Management.** SBOMTool delivers the highest component detection precision (93.67%) and enriches every entry with CPE & PURL metadata, but its dependency coverage is **0%**. To surface transitive CVEs, Cdxgen performs the 71.16% dependency detection recall, and its native PURL output can be integrated directly with *Dependency-Check* [10] or Anchore *Grype* [11]. ❸ **Licence Compliance.** For legal audits requiring rich SPDX licence detail, SPDXGen produces SPDX 2.2 JSON with component, licence, copyright, and NTIA's minimum elements; alternatively, Cdxgen can generate auto-populated licence data from package registries. ❹ **Regulatory & Procurement Requirements.** SBOMTool (SPDX 3.0) and Cdxgen (CycloneDX 1.5/1.6) already emit artifacts that satisfy the U.S. NTIA's Elements list and Executive Order 14028 submission formats without post-processing. ❺ **Third-Party Risk Assessment.** In a Java application with deep transitive graphs, Cdxgen provides indirect edges with the parameter `-profile threat-modeling` and exports a Graph-JSON extension consumable by Dependency-Track, Neo4j[12], or RiskIQ[13]. When only binary archives are available, OWASP *Dependency-Check* can still generate a coarse-grained SBOM, which may support preliminary discussions of supply chain risks.

Balancing Time Efficiency and Detection Accuracy. ❶ **Accuracy is critical for SBOM tools.** A fast but inaccurate tool could lead to misleading or incomplete results. ❷ **Deep-Scanning Tools for Compliance-Driven Scenarios.** Cdxgen incurs a long average runtime of 317.66s but achieves the best dependency detection F1-score according to our evaluation. This depth of analysis is critical in security-sensitive scenarios (e.g., healthcare or finance), where detecting more transitive components helps mitigate compliance and breach risks. ❸ **Balanced Solutions for Medium-Scale Projects.** SPDXGen (Time: 29.88s, Component Detection F1-score: 74.99%) provides a well-balanced trade-off between detection accuracy and runtime efficiency. This tool is especially suitable for open-source contributions or medium-scale applications, offering practical performance without extreme compromises.

Workflow-Oriented Strategies. ❶ **Combine Fast and Accurate Scans.** Run a lightweight scanner such as `SPDXGen` (around 30s per build) on every commit to detect obvious component changes without slowing the pipeline. Schedule a thorough tool like `Cdxgen` for nightly or pre-release runs to capture transitive dependencies and generate a high-quality SBOM. ❷ **Enterprise Managing Large Codebases.** Distributed computing may help

[10] https://owasp.org/www-project-dependency-check/
[11] https://github.com/anchore/grype
[12] https://github.com/neo4j/neo4j
[13] https://www.riskiq.com/wp-content/uploads/2020/06/RiskIQ-Company-Overview.pdf

Just Accepted

to mitigate CDXGEN's long runtime. ❸ **Security Teams Requiring Full Coverage.** For systems with complex dependency structures (e.g., microservices or plugin-based applications), choose CDXGEN to ensure complete detection even if it runs slower to meet compliance and risk control requirements.

Data Source and Dependency Resolution Considerations. Tools that combine multiple data sources (such as manifest file inspection, file system scanning, and build log analysis) tend to produce more complete SBOMs. For example, CDXGEN's hybrid static-dynamic approach enables deeper dependency resolution. In complex projects, users prefer tools that support Dependency scope filtering (e.g., `test` vs. `runtime`) and Graph-based resolution mechanisms. Such capabilities improve precision but inherently increase processing time due to the complexity of resolving transitive dependencies at scale.

Summary. This section offers practical guidance on selecting and applying open-source SBOM tools based on specific user scenarios. It classifies tools according to typical use cases (such as component-centric workflows, dependency-intensive projects, and archive-based system analysis) and compares them in terms of detection accuracy, runtime efficiency, and suitability for CI/CD pipelines. The discussion emphasizes the trade-offs between speed and detection depth, advocates for hybrid scanning strategies, and highlights the value of integrating multiple data sources to generate more complete and reliable SBOMs in complex software environments.

7.1.2 From the perspective of tool developers. We identify fundamental limitations of current SBOM tools across different component import methods and propose actionable suggestions for tool developers. Our goal is to guide future tool refinement toward better completeness, accuracy, and usability.

Enhancing the Detection Capabilities of SBOM Tools. ❶ Limitations of Overreliance on Manifest Files. Many SBOM tools primarily rely on high-level build manifests (e.g., `pom.xml`) for component and dependency detection. However, as prior studies have shown [72, 100], such manifests may be outdated or incomplete. Consequently, relying solely on them often leads to inflated or inaccurate dependency lists. Tool developers should consider incorporating lightweight build trace analysis (e.g., parsing Maven CLI logs or using build caches) to present the actual runtime dependency graph. ❷ **Need for Broader Import Methods Consideration**. Our study reveals that current SBOM tools struggle to detect components introduced via Source Code Import (SCI) and Dynamic Loading (DL). A contributing factor is the lack of evaluation datasets covering these cases. By constructing a benchmark that captures different import methods, we demonstrate that tools lacking file-level or pattern-based inspection often miss critical components. Tool supporters should consider adding detection ability for SCI and DL to augment manifest-based methods. ❸ **Module-Aware Dependency Parsing**. We observe that tools like SYFT flatten multi-module Maven projects into a single unified artifact, ignoring module boundaries and parent–child dependency hierarchies. For example, if a dependency is declared only in `module-A/pom.xml`, it may be attributed incorrectly to the entire project. This leads to misalignment with module-scoped ground truth and reduces matching accuracy. Tools should either generate SBOMs per module or include module metadata to preserve project structure.

Enhancing Result Attribution and Trust Transparency. ❶ Component Attribution Transparency. To increase the trustworthiness of SBOM outputs, we recommend that tools tag each detected component with attribution metadata, including where it was found (e.g., manifest file, classpath scan, archive unpack), how it was identified (e.g., name matching, hash comparison), and how confident the tool is in that identification (e.g., high, medium, or low confidence). Such tagging supports downstream validation and risk assessment, particularly in large and security-critical projects. ❷ **Highlight Detection Weaknesses in Outputs.** Tools should clearly present unparsed modules or skipped paths (e.g., unresolved dynamic imports) rather than failing silently. This transparency informs users of blind spots and guides remediation.

Toward Comprehensive and Benchmark-Guided Evaluation Improvements. Benchmark on Realistic Component Import Methods. Our benchmark includes not only manifest-based dependencies but also SCI

Just Accepted

and DL cases. We encourage tool developers to evaluate against such comprehensive datasets to uncover latent weaknesses.

Summary. Tool developers should evolve beyond rigid manifest-based workflows toward more flexible, modular, and context-aware detection strategies. Supporting module granularity, diverse input types, and transparent attribution can significantly improve tool accuracy and user trustworthiness.

7.1.3 From the perspective of researchers. There is a growing need to systematically investigate diverse software component import methods, which is an area underexplored in prior studies. Our work introduces a benchmark explicitly designed to assess tool effectiveness across such import methods, including dynamic loading and source code import. This benchmark also serves as a flexible foundation for evaluating SCA tools more broadly.

Toward Practical and Reproducible Evaluation Standards. ❶ Develop Grounded and Scalable Benchmarks. Most prior evaluations rely on projects, which fail to reflect the complex scenarios of real-world software supply chains. Future research should construct large-scale, diverse datasets that include varied dependency scopes, modular structures, and different import methods. This supports reproducible evaluation and comparison. **❷ Reflect Real-World Import Diversity.** Our benchmark highlights the importance of supporting overlooked import methods such as SCI and DL. Researchers should incorporate such methods into evaluation frameworks to reveal tool blind spots.

Investigating SBOM Utility in Downstream Tasks. ❶ Analyze the Impact of SBOM Completeness. There is a need to quantify the impact of SBOM quality on downstream tasks such as vulnerability detection, license compliance, and supply chain risk analysis. Establishing this connection would help researchers assess the marginal benefits of advanced detection techniques and justify their added complexity. **❷ Connect Detection Quality with Risk Models.** Beyond completeness, future studies should explore how detection accuracy and attribution granularity (e.g., confidence level, source of detection) influence risk modeling. This can bridge the gap between low-level detection strategies and high-level policy enforcement.

Designing Advanced SBOM Construction Techniques. ❶ Explore Hybrid Detection Strategies. Given the current tools' limitations in handling indirect or runtime component import, researchers are encouraged to design hybrid approaches combining static analysis (e.g., dependency graph resolution, pattern scanning) with lightweight dynamic profiling techniques that monitor component behavior during execution. Such techniques could significantly improve detection recall, particularly for runtime-resolved dependencies. **❷ Standardize Attribution Metadata for Tool Comparison.** Our ground truth analysis reveals a lack of transparency in component provenance across tools. Future work should introduce a standardized schema that clearly records where a component was detected (e.g., manifest file, archive), how it was identified (e.g., name matching, hash comparison), and the tool's confidence in that identification. Such a schema would enable more precise validation of SBOM outputs and support fair comparisons between different SBOM generation tools.

Summary. Researchers play a vital role in advancing SBOM analysis by developing practical benchmarks, investigating how SBOM quality affects downstream utility, and designing detection methods that are both hybrid and transparent. Our findings provide a foundation for future efforts to enhance the reliability, applicability, and accountability of SBOM techniques.

7.2 Threats to Validity

Internal Validity. The use of Maven as the primary build and dependency management tool brings inherent constraints, particularly with dependency redundancy issues [31, 90, 103]. However, we have chosen not to address this issue in the current SBOM tools to respect the development autonomy and integrity of the projects. Consequently, our study did not specifically tackle these Maven-related challenges, which may lead to some biases in our results.

Just Accepted

External Validity. Some methods outlined in Table 5 were not considered in this study, which may introduce some biases in our evaluation. For instance, to identify components imported via source code, we focused solely on checking the submodules used within the projects. We did not conduct a broader comparison of these projects against the current open-source software source code, which may limit the comprehensiveness of our component detection analysis.

Construct validity. The threat to construct validity mainly lies in the adopted metrics in our study. To reduce this threat, we follow prior works [118, 120] to evaluate with multiple widely-used metrics, i.e., TP, FP, FN, Precision, Recall, and F1-score.

8 CONCLUSION AND FUTURE WORK

In this work, we comprehensively evaluate six state-of-the-art free and open-source SBOM tools from the functionality (i.e., tool operational process and data fields), and practical effectiveness and efficiency in generating SBOMs. We conduct a first evaluation benchmark with three typical third-party component import scenarios with compilable datasets and precise ground truth. Then, we measure the performance of SBOM tools on our conducted benchmark and analyze the characteristics and problems in current SBOM tools. We also discuss lessons learned from different perspectives, such as tool supporters and users. We believe our research can provide the community with a clear viewpoint on this direction and inspire future researchers to find more creative ideas in this area. In the future, we plan to develop an SBOM tool that addresses these situations based on our findings and to investigate the vulnerability detection and license compliance capabilities of SBOM tools.

9 DATA AVAILABILITY

The replication package of our work is publicly available at [11].

ACKNOWLEDGMENTS

This research is supported by the National Key R&D Program of China (No. 2024YFB4506400).

REFERENCES

[1] 2024. Artillery. https://github.com/artilleryio/artillery Accessed: September 16, 2025.
[2] 2024. Hyperledger Fabric SDK for Java. https://github.com/hyperledger/fabric-sdk-java Accessed: September 16, 2025.
[3] 2024. Jemalloc. https://github.com/jemalloc/jemalloc.git Accessed: September 16, 2025.
[4] 2024. Maven Daemon (MVND). https://github.com/apache/maven-mvnd Accessed: September 16, 2025.
[5] 2024. QuestDB. https://github.com/questdb/questdb Accessed: September 16, 2025.
[6] 2024. RocketMQ Spring. https://github.com/apache/rocketmq-spring Accessed: September 16, 2025.
[7] 2024. SIMDJSON. https://github.com/simdjson/simdjson.git Accessed: September 16, 2025.
[8] Rabe Abdalkareem, Suhaib Mujahid, Emad Shihab, and Juergen Rilling. 2019. Which commits can be CI skipped? *IEEE Transactions on Software Engineering* 47, 3 (2019), 448–463.
[9] Hussein Alrubaye, Mohamed Wiem Mkaouer, and Anthony Peruma. 2019. Variability in library evolution: An exploratory study on open-source java libraries. In *Software Engineering for Variability Intensive Systems*. Auerbach Publications, 295–320.
[10] Anchore. 2024. Syft. Retrieved Oct 31, 2024. from https://github.com/anchore/syft
[11] Anonymous. 2025. TosemRepo. Retrieved Jan 31, 2025. from https://github.com/RebeWu/TOSEM25
[12] Arushi Arora, Virginia Wright, and Christina Garman. 2022. Strengthening the security of operational technology: Understanding contemporary bill of materials. *Journal of Critical Infrastructure Policy* 3, 1 (2022), 111–135.
[13] Arushi Arora, Virginia L Wright, and Christina Garman. 2022. *SoK: A Framework for and Analysis of Software Bill of Materials Tools*. Technical Report. Idaho National Laboratory (INL), Idaho Falls, ID (United States).
[14] Sushil Bajracharya, Joel Ossher, and Cristina Lopes. 2014. Sourcerer: An infrastructure for large-scale collection and analysis of open-source code. *Science of Computer Programming* 79 (2014), 241–259.
[15] Musard Balliu, Benoit Baudry, Sofia Bobadilla, Mathias Ekstedt, Martin Monperrus, Javier Ron, Aman Sharma, Gabriel Skoglund, César Soto-Valero, and Martin Wittlinger. 2023. Challenges of producing software bill of materials for java. *IEEE Security & Privacy* (2023).

Just Accepted

[16] Iain Barclay, Alun Preece, Ian Taylor, and Dinesh Verma. 2021. Towards Traceability in Data Ecosystems using a Bill of Materials Model. (2021).
[17] Stefan Bellon, Rainer Koschke, Giulio Antoniol, Jens Krinke, and Ettore Merlo. 2007. Comparison and evaluation of clone detection tools. *IEEE Transactions on software engineering* 33, 9 (2007), 577–591.
[18] Amine Benelallam, Nicolas Harrand, César Soto-Valero, Benoit Baudry, and Olivier Barais. 2019. The maven dependency graph: a temporal graph-based representation of maven central. In *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*. IEEE, 344–348.
[19] Pavan Sai Beri and Arun Mishra. 2019. Dynamic software component authentication for autonomous systems using slack space. In *2019 3rd International Conference on Trends in Electronics and Informatics (ICOEI)*. IEEE, 905–910.
[20] Tingting Bi, Boming Xia, Zhenchang Xing, Qinghua Lu, and Liming Zhu. 2024. On the way to sboms: Investigating design issues and solutions in practice. *ACM Transactions on Software Engineering and Methodology* 33, 6 (2024), 1–25.
[21] bitbucket. 2024. bitbucket. Retrieved Oct 31, 2024. from https://bitbucket.org
[22] James Bonacci and Reese Martin. 2024. Software Bill of Materials (SBOM) Approach to IoT Security Vulnerability Assessment. In *International Conference on Information Technology-New Generations*. Springer, 57–62.
[23] Laura Bottner, Artur Hermann, Jeremias Eppler, Thomas Thüm, and Frank Kargl. 2023. Evaluation of Free and Open Source Tools for Automated Software Composition Analysis. In *Proceedings of the 7th ACM Computer Science in Cars Symposium*. 1–11.
[24] Security Boulevard. [n. d.]. SBOM Security in 2023: Top 5 Things You Must Know. https://securityboulevard.com/2023/03/sbom-security-in-2023-top-5-things-you-must-know-as-a-cybersecurity-professional-2/
[25] Premek Brada and Kamil Jezek. 2015. Repository and meta-data design for efficient component consistency verification. *Science of Computer Programming* 97 (2015), 349–365.
[26] Gianluigi Caldiera and Victor R. Basili. 1991. Identifying and qualifying reusable software components. *Computer* 24, 2 (1991), 61–70.
[27] Seth Carmody, Andrea Coravos, Ginny Fahs, Audra Hatch, Janine Medina, Beau Woods, and Joshua Corman. 2021. Building resilient medical technology supply chains with a software bill of materials. *NPJ Digital Medicine* 4, 1 (2021), 1–6.
[28] Luis Paulo da Silva Carvalho. 2021. Identifying and analyzing software concerns from third-party components' metadata. (2021).
[29] Continuity Central. [n. d.]. 2022 Global State of Enterprise Risk Oversight. https://erm.ncsu.edu/resource-center/2022-global-state-of-enterprise-risk-oversight/
[30] Brian Ka Chan. 2017. Artificial Intelligence Bill of Materials (AIBOM). https://minddata.org/bill-of-artificial-intelligence-materials-boaimBrian-Ka-Chan-AI. Accessed: 2024-10-16.
[31] Ching-Chi Chuang, Luis Cruz, Robbert van Dalen, Vladimir Mikovski, and Arie van Deursen. 2022. Removing dependencies from large software projects: are you really sure?. In *2022 IEEE 22nd International Working Conference on Source Code Analysis and Manipulation (SCAM)*. IEEE, 105–115.
[32] William Gemmell Cochran. 1977. *Sampling techniques*. john wiley & sons.
[33] CVE Program. 1999. Common Vulnerabilities and Exposures (CVE) Website. https://www.cve.org/. Accessed 16 Jun 2025.
[34] Cybersecurity and Infrastructure Security Agency, National Security Agency, and Office of the Director of National Intelligence. 2024. *Securing the Software Supply Chain: Recommended Practices for Software Bill of Materials (SBOM) Consumption*. Technical Report. Enduring Security Framework Working Group. https://www.cisa.gov/sites/default/files/2024-08/SECURING_THE_SOFTWARE_SUPPLY_CHAIN_RECOMMENDED_PRACTICES_FOR_SOFTWARE_BILL_OF_MATERIALS_CONSUMPTION-508.pdf Part 3 of the ESF "Securing the Software Supply Chain" series.
[35] Cyclonedx. 2021. Cyclonedx. https://cyclonedx.org/
[36] CycloneDX. 2022. Hardware Bill of Materials (HBOM). https://github.com/CycloneDX/bom-examples/tree/master/HBOM. Accessed: 2024-10-16.
[37] CycloneDX. 2022. Operations Bill of Materials (OBOM). https://github.com/CycloneDX/bom-examples/tree/master/OBOM. Accessed: 2024-10-16.
[38] CycloneDX. 2024. CycloneDX Generator. Retrieved Oct 31, 2024. from https://github.com/CycloneDX/cdxgen
[39] Gregorio Dalia, Corrado Aaron Visaggio, Andrea Di Sorbo, and Gerardo Canfora. 2024. SBOM Ouverture: What We Need and What We Have. In *Proceedings of the 19th International Conference on Availability, Reliability and Security*. 1–9.
[40] Andreas Dann, Henrik Plate, Ben Hermann, Serena Elisa Ponta, and Eric Bodden. 2021. Identifying challenges for oss vulnerability scanners-a study & test suite. *IEEE Transactions on Software Engineering* 48, 9 (2021), 3613–3625.
[41] National Vulnerability Database. 2021. CVE-2021-44228: Apache Log4j Vulnerability. https://nvd.nist.gov/vuln/detail/CVE-2021-44228 Accessed: Oct 31, 2024..
[42] DevOps.com. [n. d.]. DevOps World 2022: Using SBOMs to Secure the Software Supply Chain. https://devops.com/devops-world-2022-using-sboms-to-secure-the-software-supply-chain/
[43] Jens Dietrich, Shawn Rasheed, and Alexander Jordan. 2023. On the Security Blind Spots of Software Composition Analysis. *arXiv preprint arXiv:2306.05534* (2023).

Just Accepted

[44] Xinxing Ding, Feng Zhao, Lijuan Yan, and Xiaodong Shao. 2019. The Method of Building SBOM Based on Enterprise Big Data. In *2019 3rd International Conference on Electronic Information Technology and Computer Engineering (EITCE)*. IEEE, 1224–1228.
[45] FOSSA. [n. d.]. SBOM Requirements in the EU's Cyber Resilience Act. https://fossa.com/blog/sbom-requirements-cra-cyber-resilience-act/
[46] Apache Software Foundation. 2024. Apache Maven Project. Retrieved Oct 7, 2024. from https://maven.apache.org/
[47] Apache Software Foundation. 2024. Maven CLI. Retrieved September 16, 2025from https://maven.apache.org/ref/3.6.3/maven-embedder/cli.html
[48] Linux Foundation. 2022. SPDX 2.3 Specification. https://spdx.dev/specifications/. Accessed: 2024-10-16.
[49] OWASP Foundation. 2023. OWASP Vulnerability Management Guide. https://owasp.org/www-project-vulnerability-management-guide/. Accessed: 2024-10-16.
[50] OWASP Foundation. 2024. OWASP Dependency Track. Retrieved Oct 31, 2024. from https://owasp.org/www-project-dependency-track/
[51] The Linux Foundation. 2024. SPDX specification. Retrieved Oct 31, 2024. from https://spdx.dev/use/specifications/
[52] Free Software Foundation. 2007. GNU General Public License, Version 3. https://www.gnu.org/licenses/gpl-3.0.en.html. Accessed 16 Jun 2025.
[53] Git. 2023. Git Tools - Submodules. https://www.git-scm.com/book/en/v2/Git-Tools-Submodules
[54] GitHub. 2024. GitHub. Retrieved Oct 31, 2024. from https://github.com/
[55] Gradle. 2024. Gradle Build Tool. Retrieved Oct 31, 2024. from https://maven.apache.org/
[56] Andreas Halbritter and Dominik Merli. 2024. Accuracy Evaluation of SBOM Tools for Web Applications and System-Level Software. In *Proceedings of the 19th International Conference on Availability, Reliability and Security*.
[57] Hideaki Hata, Christoph Treude, Raula Gaikovina Kula, and Takashi Ishio. 2019. 9.6 million links in source code comments: Purpose, evolution, and decay. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 1211–1221.
[58] Yikun Hu, Yuanyuan Zhang, Juanru Li, and Dawu Gu. 2017. Binary code clone detection across architectures and compiling configurations. In *2017 IEEE/ACM 25th International Conference on Program Comprehension (ICPC)*. IEEE, 88–98.
[59] Kaifeng Huang, Bihuan Chen, Congying Xu, Ying Wang, Bowen Shi, Xin Peng, Yijian Wu, and Yang Liu. 2022. Characterizing usages, updates and risks of third-party libraries in Java projects. *Empirical Software Engineering* 27, 4 (2022), 90.
[60] Nasif Imtiaz, Seaver Thorn, and Laurie Williams. 2021. A comparative study of vulnerability reporting by software composition analysis tools. In *Proceedings of the 15th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. 1–11.
[61] GitLab Inc. [n. d.]. Dependency scanning by using SBOM. https://docs.gitlab.com/user/application_security/dependency_scanning/dependency_scanning_sbom/. accessed 4 Jun 2025.
[62] Open Source Initiative. 2023. The License Review Process. https://opensource.org/licenses/review-process/. Accessed: 2024-10-16.
[63] Takashi Ishio, Raula Gaikovina Kula, Tetsuya Kanda, Daniel M German, and Katsuro Inoue. 2016. Software ingredients: Detection of third-party component reuse in java software release. In *Proceedings of the 13th International Conference on Mining Software Repositories*. 339–350.
[64] Jianxin Jiao, Mitchell M Tseng, Qinhai Ma, and Yi Zou. 2000. Generic bill-of-materials-and-operations for high-variety production management. *Concurrent Engineering* 8, 4 (2000), 297–321.
[65] Omercan Kağızmandere and Halil Arslan. 2024. Vulnerability analysis based on Software Bill of Materials (SBOM): A model proposal for automated vulnerability scanning for CI/CD pipelines. *International Journal of Information Security Science* 13, 2 (2024).
[66] Byoungchul Kim, Kyeonghwan Lim, Seong-Je Cho, and Minkyu Park. 2019. Romadroid: A robust and efficient technique for detecting android app clones using a tree structure and components of each app's manifest file. *IEEE Access* 7 (2019), 72182–72196.
[67] Berend Kloeg, Aaron Yi Ding, Sjoerd Pellegrom, and Yury Zhauniarovich. 2024. Charting the Path to SBOM Adoption: A Business Stakeholder-Centric Approach. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*. 1770–1783.
[68] CI/CD Best Practices Knowledgebase. [n. d.]. CI/CD Best Practices. https://cicdbestpractises.com/. Accessed: 2024-10-16.
[69] Raula Gaikovina Kula, Coen De Roover, Daniel M German, Takashi Ishio, and Katsuro Inoue. 2018. A generalized model for visualizing library popularity, adoption, and diffusion within a software ecosystem. In *2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 288–299.
[70] Adrienn Lawson and Stephen Hendrick. 2023. Global Spotlight 2023: Survey-based insights into the global landscape of open source trends, sustainability challenges, and growth opportunities. https://www.linuxfoundation.org/research/world-of-open-source-global-2023
[71] Menghao Li, Wei Wang, Pei Wang, Shuai Wang, Dinghao Wu, Jian Liu, Rui Xue, and Wei Huo. 2017. Libd: Scalable and precise third-party library detection in android markets. In *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. IEEE, 335–346.
[72] Yue Li, Tian Tan, and Jingling Xue. 2019. Understanding and analyzing java reflection. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 28, 2 (2019), 1–50.
[73] Zhuang Liu, Xing Hu, Jiayuan Zhou, and Xin Xia. 2025. From Industrial Practices to Academia: Uncovering the Gap in Vulnerability Research and Practice. In *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. IEEE, 129–141.

Just Accepted

[74] Massachusetts Institute of Technology. 1988. MIT License. https://opensource.org/licenses/MIT. Accessed 16 Jun 2025.
[75] Maven. 2024. MVN Repository. https://mvnrepository.com/repos/central Accessed: Oct 31, 2024..
[76] Apache Maven. 2024. Maven Central. Retrieved Oct 31, 2024. from https://mvnrepository.com/repos/central
[77] Shane McIntosh, Meiyappan Nagappan, Bram Adams, Audris Mockus, and Ahmed E Hassan. 2015. A large-scale empirical study of the relationship between build technology and build maintenance. *Empirical Software Engineering* 20 (2015), 1587–1633.
[78] Microsoft. 2024. SBOM Tool. Retrieved Oct 31, 2024. from https://github.com/microsoft/sbom-tool
[79] National Institute of Standards and Technology. 2012. National Vulnerability Database. http://web.nvd.nist.gov/. Accessed 16 Jun 2025.
[80] Sabato Nocera, Simone Romano, Massimiliano Di Penta, Rita Francese, and Giuseppe Scanniello. 2023. Software bill of materials adoption: a mining study from GitHub. In *2023 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 39–49.
[81] Yusuf Sulistyo Nugroho, Hideaki Hata, and Kenichi Matsumoto. 2020. How different are different diff algorithms in Git? Use–histogram for code changes. *Empirical Software Engineering* 25 (2020), 790–823.
[82] Chinenye Okafor, Taylor R Schorlemmer, Santiago Torres-Arias, and James C Davis. 2022. Sok: Analysis of software supply chain security by establishing secure design properties. In *Proceedings of the 2022 ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses*. 15–24.
[83] opensbom generator. 2024. SPDXGen. Retrieved Oct 31, 2024. from https://github.com/opensbom-generator/spdx-sbom-generator
[84] Marc Oriol, Carlos Müller, Jordi Marco, Pablo Fernandez, Xavier Franch, and Antonio Ruiz-Cortés. 2023. Comprehensive assessment of open source software ecosystem health. *Internet of Things* 22 (2023), 100808.
[85] Joel Ossher, Sushil Bajracharya, and Cristina Lopes. 2010. Automated dependency resolution for open source software. In *2010 7th IEEE Working Conference on Mining Software Repositories (MSR 2010)*. 130–140. https://doi.org/10.1109/MSR.2010.5463346
[86] Wataru Otoda, Tetsuya Kanda, Yuki Manabe, Katsuro Inoue, and Yoshiki Higo. 2024. SBOM Challenges for Developers: From Analysis of Stack Overflow Questions. In *2024 IEEE/ACIS 22nd International Conference on Software Engineering Research, Management and Applications (SERA)*. IEEE, 43–46.
[87] Eric O'Donoghue, Brittany Boles, Clemente Izurieta, and Ann Marie Reinhold. 2024. Impacts of Software Bill of Materials (SBOM) Generation on Vulnerability Detection. In *2024 ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses*.
[88] Shengyi Pan, Lingfeng Bao, Jiayuan Zhou, Xing Hu, Xin Xia, and Shanping Li. 2024. Unveil the mystery of critical software vulnerabilities. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. 138–149.
[89] Henrik Plate, Serena Elisa Ponta, and Antonino Sabetta. 2015. Impact assessment for vulnerabilities in open-source software libraries. In *2015 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 411–420.
[90] Serena Elisa Ponta, Wolfram Fischer, Henrik Plate, and Antonino Sabetta. 2021. The used, the bloated, and the vulnerable: Reducing the attack surface of an industrial application. In *2021 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 555–558.
[91] Md Fazle Rabbi, Arifa Islam Champa, Costain Nachuma, and Minhaz Fahim Zibran. 2024. Sbom generation tools under microscope: A focus on the npm ecosystem. In *Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing*. 1233–1241.
[92] T. Ravichandran and Marcus A. Rothenberger. 2003. Software reuse strategies and component markets. *Commun. ACM* 46, 8 (aug 2003), 109–114. https://doi.org/10.1145/859670.859678
[93] Pasquale Salza, Fabio Palomba, Dario Di Nucci, Andrea De Lucia, and Filomena Ferrucci. 2020. Third-party libraries in mobile apps: When, how, and why developers update them. *Empirical Software Engineering* 25 (2020), 2341–2377.
[94] Aqua Security. 2024. Trivy. Retrieved Oct 31, 2024. from https://github.com/aquasecurity/trivy
[95] César Soto-Valero, Nicolas Harrand, Martin Monperrus, and Benoit Baudry. 2021. A comprehensive study of bloated dependencies in the maven ecosystem. *Empirical Software Engineering* 26, 3 (2021), 45.
[96] Trevor Stalnaker, Nathan Wintersgill, Oscar Chaparro, Massimiliano Di Penta, Daniel M German, and Denys Poshyvanyk. 2024. Boms away! inside the minds of stakeholders: A comprehensive study of bills of materials for software systems. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13.
[97] Trevor Wayne Stalnaker. 2023. *A Comprehensive Study of Bills of Materials for Software Systems.* Master's thesis. The College of William and Mary.
[98] Jeremiah Trent Stoddard, Michael Adam Cutshaw, Tyler Williams, Allan Friedman, and Justin Murphy. 2023. *Software Bill of Materials (SBOM) Sharing Lifecycle Report.* Technical Report. Idaho National Laboratory (INL), Idaho Falls, ID (United States).
[99] Siddharth Subramanian, Laura Inozemtseva, and Reid Holmes. 2014. Live API documentation. In *Proceedings of the 36th international conference on software engineering*. 643–652.
[100] Li Sui, Jens Dietrich, Michael Emery, Shawn Rasheed, and Amjed Tahir. 2018. On the soundness of call graph construction in the presence of dynamic language features-a benchmark and tool evaluation. In *Programming Languages and Systems: 16th Asian Symposium, APLAS 2018, Wellington, New Zealand, December 2–6, 2018, Proceedings 16*. Springer, 69–88.
[101] Wei Tang, Ping Luo, Jialiang Fu, and Dan Zhang. 2020. Libdx: A cross-platform and accurate system to detect third-party libraries in binary code. In *2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 104–115.

Just Accepted

[102] Wei Tang, Zhengzi Xu, Chengwei Liu, Jiahui Wu, Shouguo Yang, Yi Li, Ping Luo, and Yang Liu. 2022. Towards understanding third-party library dependency in C/C++ ecosystem. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–12.
[103] Núria Bruch Tarrega, Miroslav Zivkovic, Ana Oprescu, and S PCS. 2020. Measuring the Impact of Library Dependency on Maintenance.. In *SATToSE*.
[104] National Telecommunications and Information Administration. 2021. Minimum Elements for an SBOM. https://www.ntia.doc.gov/files/ntia/publications/sbom_minimum_elements_report.pdf Accessed: 2024-10-16.
[105] Theodoros Theodoropoulos, Luis Rosa, Chafika Benzaid, Peter Gray, Eduard Marin, Antonios Makris, Luis Cordeiro, Ferran Diego, Pavel Sorokin, Marco Di Girolamo, et al. 2023. Security in Cloud-Native Services: A Survey. *Journal of Cybersecurity and Privacy* 3, 4 (2023), 758–793.
[106] TIA. 2021. The Minimum Elements For a Software Bill of Materials (SBOM). https://www.ntia.doc.gov/files/ntia/publications/sbom_minimum_elements_report.pdf. Accessed: 2024-10-16.
[107] Raoul-Gabriel Urma, Alan Mycroft, and Mario Fusco. 2018. *Modern Java in Action: Lambdas, streams, functional and reactive programming*. Simon and Schuster.
[108] Andy Ju An Wang and Kai Qian. 2005. *Component-oriented programming*. John Wiley & Sons.
[109] Yafei Wu, Cong Sun, Dongrui Zeng, Gang Tan, Siqi Ma, and Peicheng Wang. 2023. {LibScan}: Towards more precise {Third-Party} library identification for android applications. In *32nd USENIX Security Symposium (USENIX Security 23)*. 3385–3402.
[110] Yulun Wu, Zeliang Yu, Ming Wen, Qiang Li, Deqing Zou, and Hai Jin. 2023. Understanding the threats of upstream vulnerabilities to downstream projects in the maven ecosystem. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1046–1058.
[111] Boming Xia, Tingting Bi, Zhenchang Xing, Qinghua Lu, and Liming Zhu. 2023. An empirical study on software bill of materials: Where we stand and the road ahead. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2630–2642.
[112] Boming Xia, Dawen Zhang, Yue Liu, Qinghua Lu, Zhenchang Xing, and Liming Zhu. 2024. Trust in software supply chains: Blockchain-enabled sbom and the aibom future. In *Proceedings of the 2024 ACM/IEEE 4th International Workshop on Engineering and Cybersecurity of Critical Systems (EnCyCriS) and 2024 IEEE/ACM Second International Workshop on Software Vulnerability*. 12–19.
[113] Yue Xiao, Dhilung Kirat, Douglas Lee Schales, Jiyong Jang, Luyi Xing, and Xiaojing Liao. 2025. JBomAudit: Assessing the Landscape, Compliance, and Security Implications of Java SBOMs. In *ISOC Network and Distributed System Security Symposium*.
[114] Narges Yousefnezhad and Andrei Costin. 2024. Understanding SBOMs in Real-World Systems–A Practical DevOps/SecOps Perspective. In *International Symposium on Business Modeling and Software Design*. Springer, 293–304.
[115] Sheng Yu, Wei Song, Xunchao Hu, and Heng Yin. 2024. On the Correctness of Metadata-based SBOM Generation: A Differential Analysis Approach. In *2024 54th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 29–36.
[116] Nusrat Zahan, Elizabeth Lin, Mahzabin Tamanna, William Enck, and Laurie Williams. 2023. Software bills of materials are required. are we there yet? *IEEE Security & Privacy* 21, 2 (2023), 82–88.
[117] Morteza Zakeri-Nasrabadi, Saeed Parsa, Mohammad Ramezani, Chanchal Roy, and Masoud Ekhtiarzadeh. 2023. A systematic literature review on source code similarity measurement and clone detection: Techniques, applications, and challenges. *Journal of Systems and Software* (2023), 111796.
[118] Xian Zhan, Tianming Liu, Yepang Liu, Yang Liu, Li Li, Haoyu Wang, and Xiapu Luo. 2021. A systematic assessment on Android third-party library detection tools. *IEEE Transactions on Software Engineering* 48, 11 (2021), 4249–4273.
[119] Lyuye Zhang, Chengwei Liu, Zhengzi Xu, Sen Chen, Lingling Fan, Lida Zhao, Jiahui Wu, and Yang Liu. 2023. Compatible remediation on vulnerabilities from third-party libraries for java projects. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2540–2552.
[120] Lida Zhao, Sen Chen, Zhengzi Xu, Chengwei Liu, Lyuye Zhang, Jiahui Wu, Jun Sun, and Yang Liu. 2023. Software composition analysis for vulnerability detection: An empirical study on Java projects. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 960–972.
[121] Chunliu Zhou, Xiaobing Liu, Fanghong Xue, Hongguang Bo, and Kai Li. 2018. Research on static service BOM transformation for complex products. *Advanced engineering informatics* 36 (2018), 146–162.

Just Accepted